Machine learning

Lecture 3

Deep learning

Inigo Bermejo
inigo.bermejo@uhasselt.be

DSI DATA SCIENCE INSTITUTE
UHASSELT

WWW.UHASSELT.BE/DSI

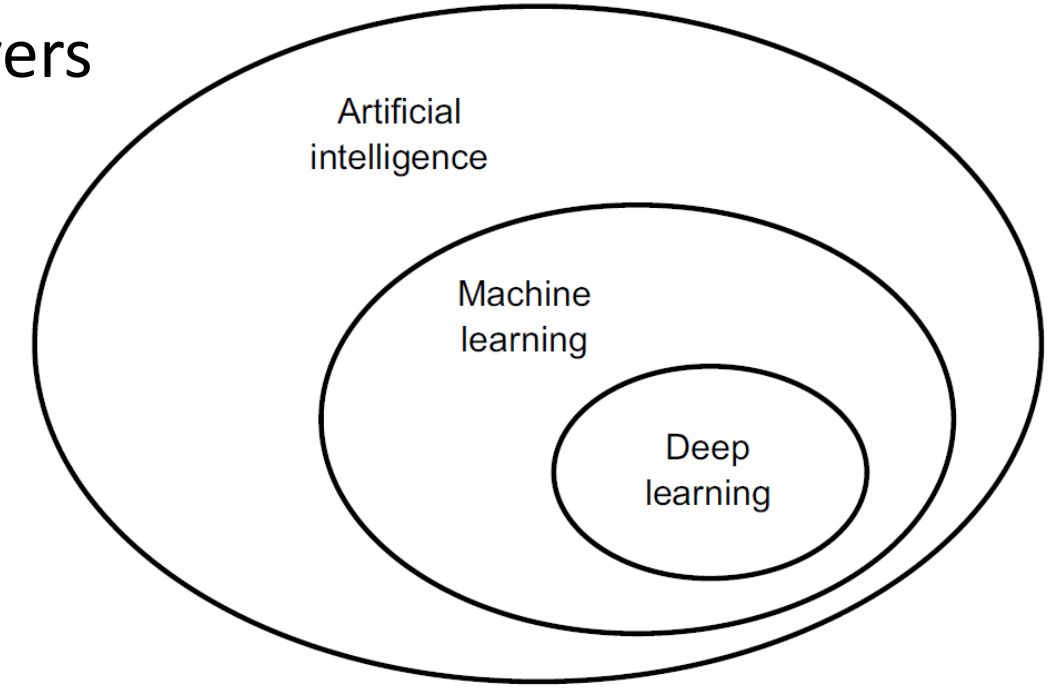
Outline

- Fundaments of deep learning
- Training neural networks
- Building neural networks
- Convolutional neural networks
- Recurrent neural networks
- When to use deep learning

What is deep learning?

What is deep learning?

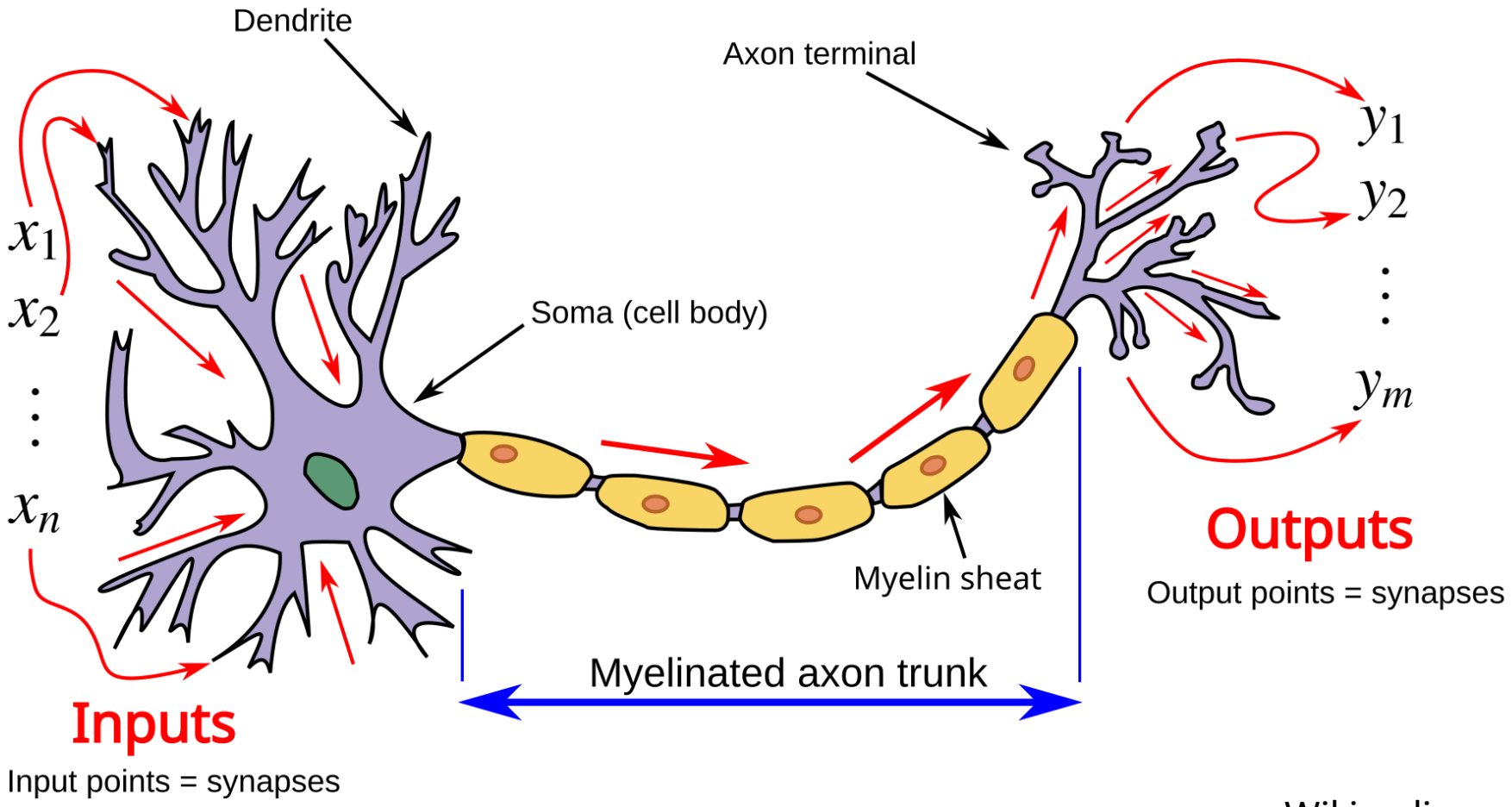
- A subfield of machine learning
- Essentially, a rebranding of neural networks
- ‘Deep’ refers to the high number of layers
- Why was a rebranding needed?
- What are neural networks?



What are (artificial) neural networks?

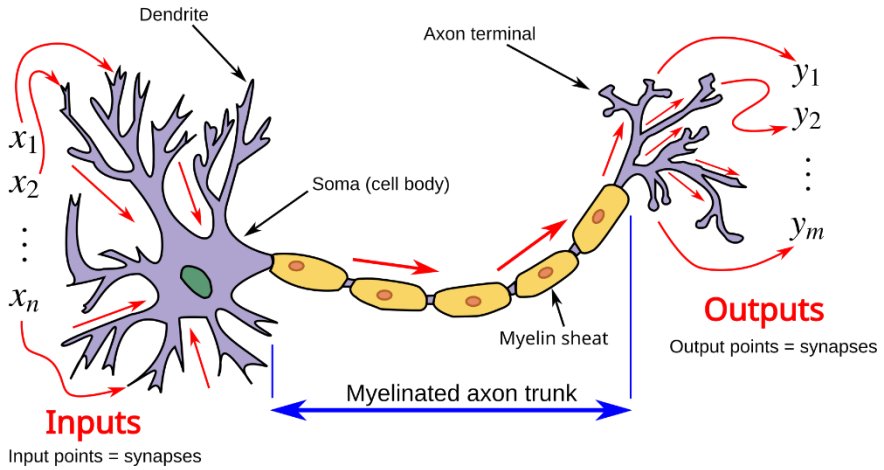
- Computational models (loosely) inspired by the brain
- Composed of interconnected artificial neurons

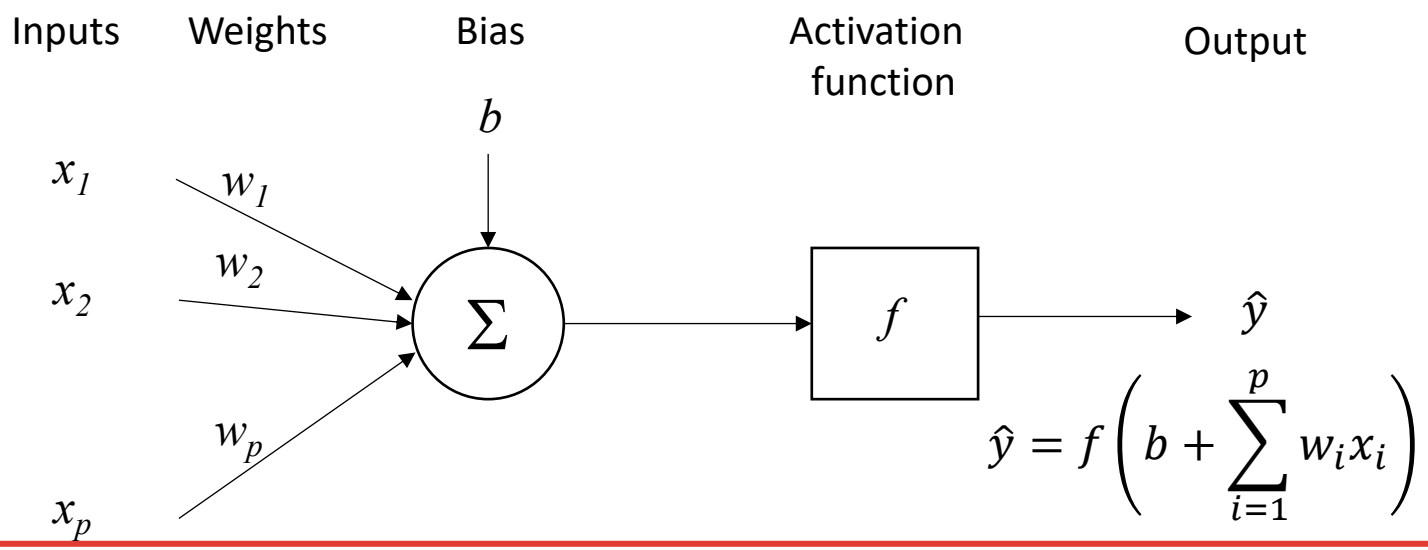
The neuron



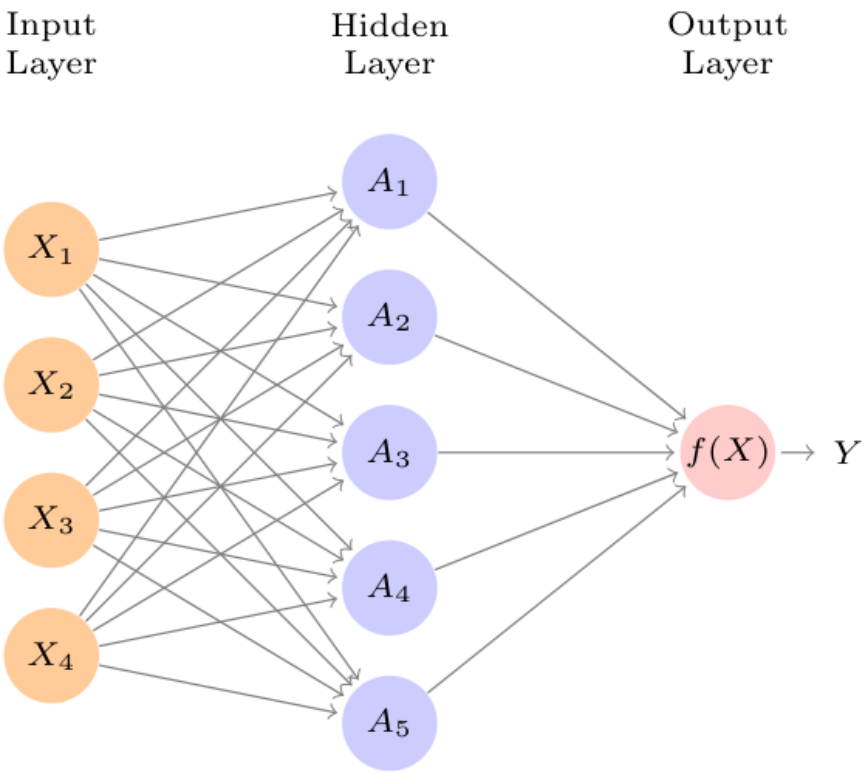
source: Wikipedia

The artificial neuron





The single layer neural network



Forward pass / propagation

Weights (Hidden layer)

$$A_k = h_k(X) = g\left(w_{k0} + \sum_{j=1}^{p} w_{kj} X_j\right)$$

Input data

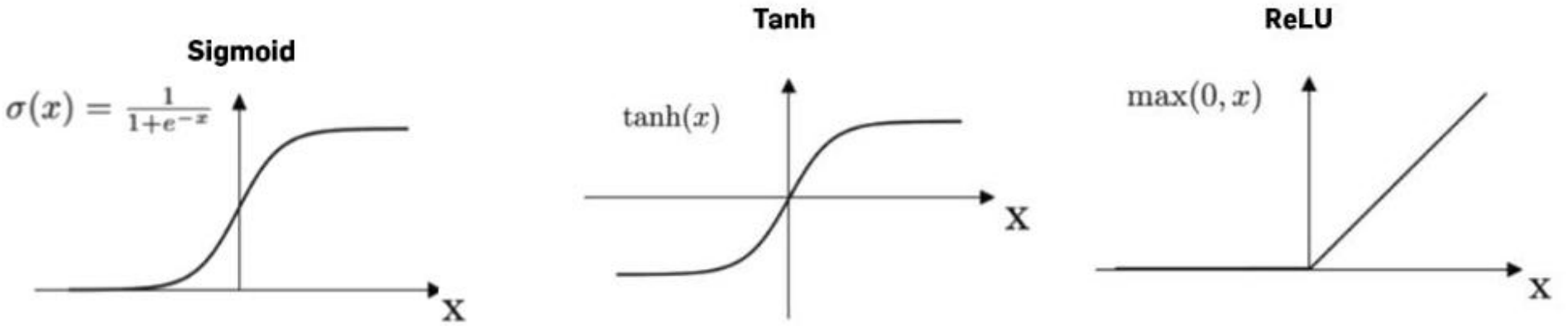
g(x) = activation function

$$f(X) = \beta_0 + \sum_{k=1}^{K} \beta_k A_k$$

Weights (Output layer)

The activation function

Non-linearity in activation functions essential to NNs

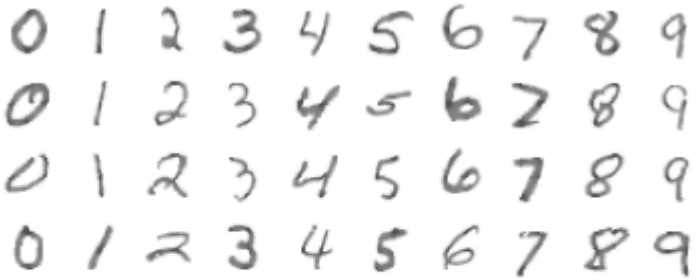


- A sequence of linear transformations is still a linear transformation
- Activation functions necessary to model non-linearity, interactions

Multilayer neural network

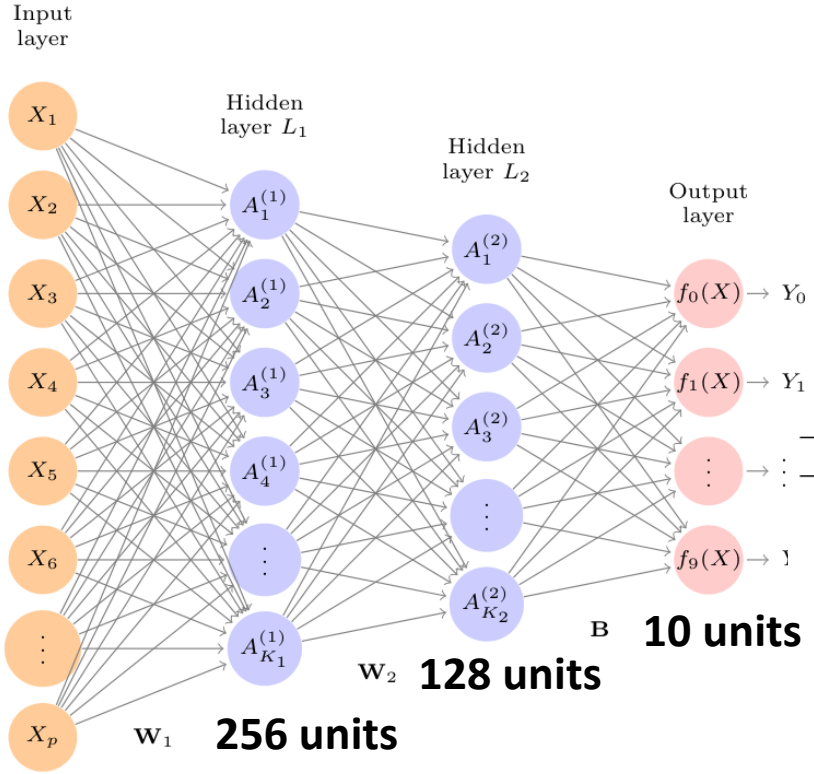
- Most functions can be approximated with one hidden layer
- However, learning is made much easier with multiple layers

MNIST handwritten digits



28x28 pixels

Labels one-hot encoding e.g 5=(0000010000)



28x28 = 784 units

256 units

128 units

10 units

Softmax activation

$$f_m(X) = \Pr(Y = m|X) = \frac{e^{Z_m}}{\sum_{\ell=0}^{9} e^{Z_\ell}},$$

Method	Test Error
Neural Network + Ridge Regularization	2.3%
Neural Network + Dropout Regularization	1.8%
Multinomial Logistic Regression	7.2%

But, how do we train neural networks?

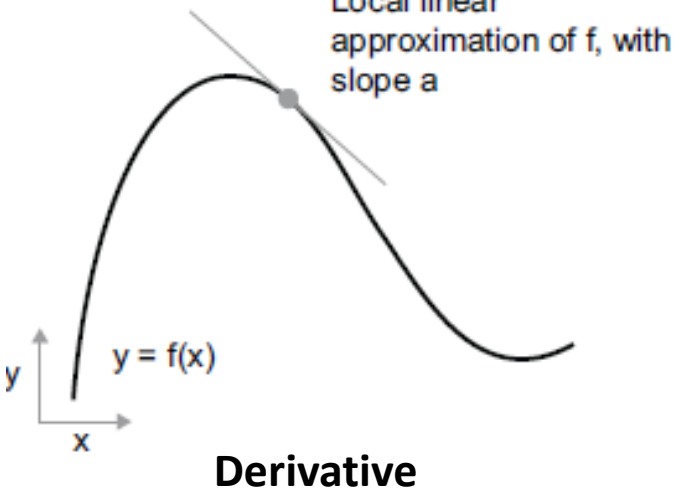
Training neural networks

The loss function

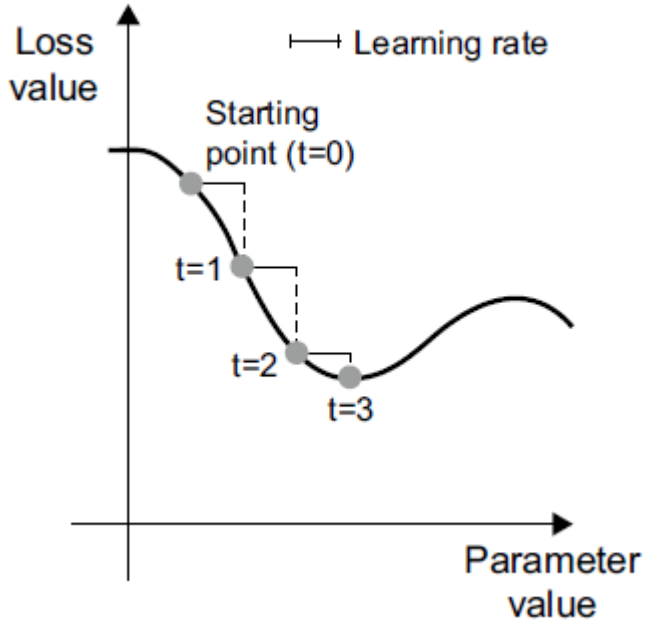
- Function that measures distance between predicted & observed
- Training a network = minimising the loss function
- Needs to be differentiable
- Examples:
  - MSE = $\frac{1}{n}\sum(y - \hat{y})^2$
  - Binary cross-entropy = $(y * \log \hat{y}) + ((1 - y) * \log(1 - \hat{y}))$

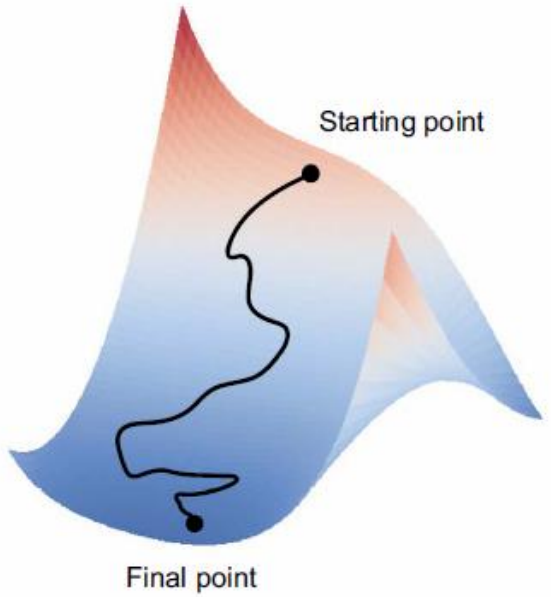
Gradient descent

- Optimisation algorithm used to train NNs
- Uses derivative (gradient) of loss function wrt weight(s)
- Tells us which way to adjust weights to reduce loss
- Iterative process: every pass of the data is an *epoch*



Derivative



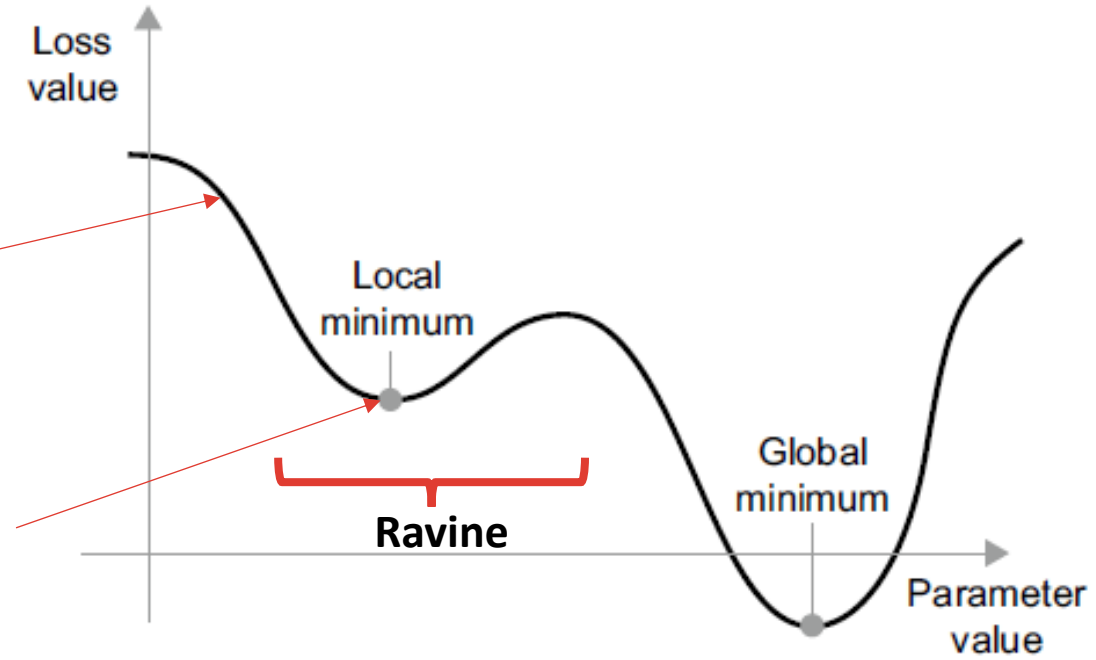


Gradient descent

Limitations of gradient descent: short sightedness

If we start here

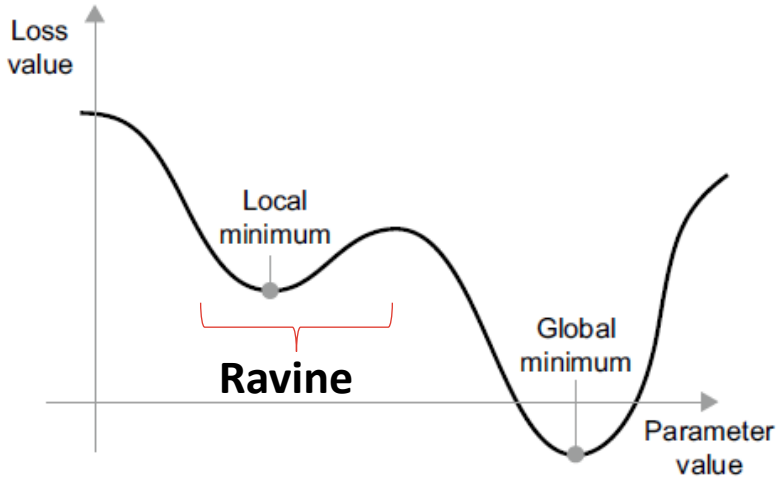
We'll be stuck there



Gradient descent

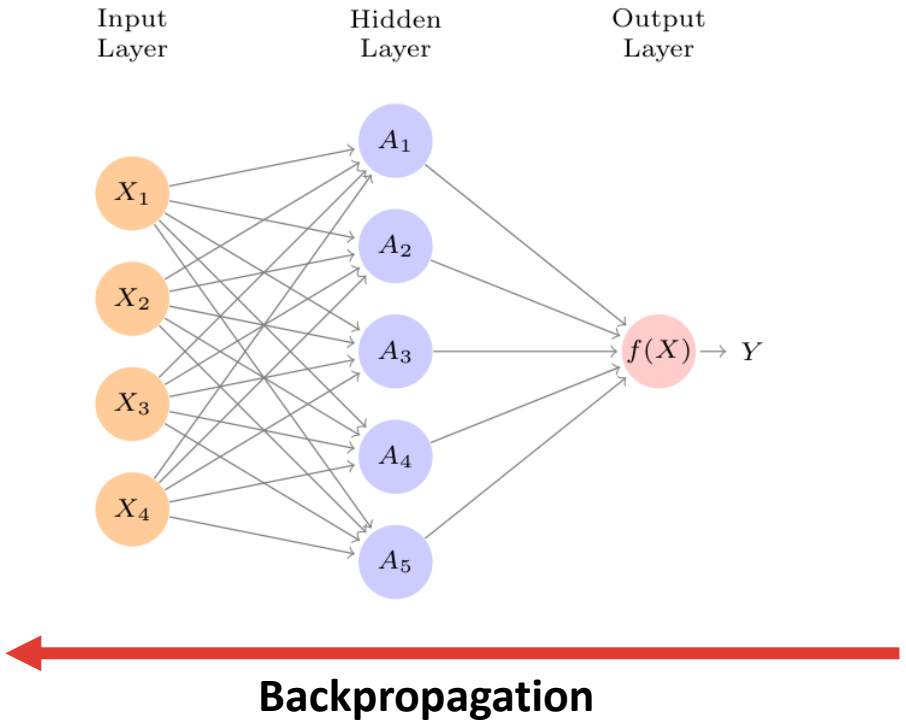
Overcoming local minima with **momentum**

- Think of optimisation as a ball
- Move not only based on current slope *but also* previous steps (velocity)
- The *optimizer* determines how weights are updated:
  - E.g. RMSProp, AdaGrad



Backpropagation

- How can we train the hidden and the input layers?
- Propagating error from output layer back to previous layers
- Compute each unit's contribution to downstream error



Backpropagation

Backpropagation in logistic regression

Goal: $\frac{\partial L}{\partial w}, \frac{\partial L}{\partial b}$

Forward pass

Linear combination

$$z = w^T x + b$$

Activation function **(sigmoid)**

$$\hat{y} = \sigma(z) \qquad \sigma(z) = \frac{1}{1 + e^{-z}}$$

Loss function (binary cross entropy)

$$L(y, \hat{y}) = -\left[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})\right]$$

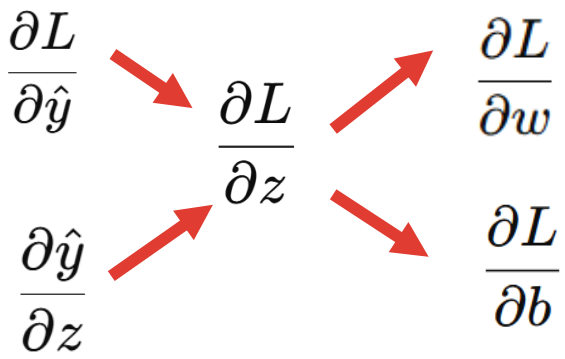
Computational graph

$$x \rightarrow z \rightarrow \hat{y} \rightarrow L$$

Backpropagation

Goal: $\frac{\partial L}{\partial w}, \frac{\partial L}{\partial b}$

Computational graph:



$$\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}$$

Backpropagation

Backpropagation in logistic regression

Goal: $\frac{\partial L}{\partial w}, \frac{\partial L}{\partial b}$

Forward pass

Linear combination

$$z = w^T x + b$$

Activation function **(sigmoid)**

$$\hat{y} = \sigma(z) \qquad \sigma(z) = \frac{1}{1+e^{-z}}$$

Loss function (binary cross entropy)

$$L(y, \hat{y}) = -\left[y \log(\hat{y}) + (1-y)\log(1-\hat{y})\right]$$

Computational graph

$$x \rightarrow z \rightarrow \hat{y} \rightarrow L$$

Backpropagation

First step, derivative of loss wrt prediction $\frac{\partial L}{\partial \hat{y}}$

$$\frac{\partial L}{\partial \hat{y}} \begin{cases} \frac{\partial}{\partial \hat{y}}\left(-y \log(\hat{y})\right) = -\frac{y}{\hat{y}} \\ \frac{\partial}{\partial \hat{y}}\left(-(1-y)\log(1-\hat{y})\right) = \frac{1-y}{1-\hat{y}} \end{cases}$$

$$\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}$$

Backpropagation

$$\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}$$

$$\frac{\partial \hat{y}}{\partial z} = \hat{y}(1-\hat{y})$$

Backpropagation in logistic regression

Goal: $\frac{\partial L}{\partial w}, \frac{\partial L}{\partial b}$

Forward pass

Linear combination

$$z = w^T x + b$$

Activation function **(sigmoid)**

$$\hat{y} = \sigma(z) \qquad \sigma(z) = \frac{1}{1+e^{-z}}$$

Loss function (binary cross entropy)

$$L(y, \hat{y}) = -\left[y \log(\hat{y}) + (1-y)\log(1-\hat{y})\right]$$

Computational graph

$$x \rightarrow z \rightarrow \hat{y} \rightarrow L$$

Backpropagation

Next, the derivative of sigmoid wrt z

$\frac{\partial \hat{y}}{\partial z}$

The derivative is

$$\sigma'(z) = \sigma(z)(1-\sigma(z))$$

Replacing $\sigma(z)$ by $\hat{y}$

$$\frac{\partial \hat{y}}{\partial z} = \hat{y}(1-\hat{y})$$

Backpropagation

$$\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}$$

$$\frac{\partial \hat{y}}{\partial z} = \hat{y}(1-\hat{y})$$

Backpropagation in logistic regression

Goal: $\frac{\partial L}{\partial w}, \frac{\partial L}{\partial b}$

Forward pass

Linear combination

$$z = w^T x + b$$

Activation function **(sigmoid)**

$$\hat{y} = \sigma(z) \qquad \sigma(z) = \frac{1}{1+e^{-z}}$$

Loss function (binary cross entropy)

$$L(y, \hat{y}) = -\left[y \log(\hat{y}) + (1-y)\log(1-\hat{y})\right]$$

Computational graph

$$x \rightarrow z \rightarrow \hat{y} \rightarrow L$$

Backpropagation

Now, we calculate the derivative of loss wrt z $\frac{\partial L}{\partial z}$

Apply the chain rule

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z}$$

Substitute the expressions

$$\begin{aligned}\frac{\partial L}{\partial z} &= \left(-\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}\right)\hat{y}(1-\hat{y}) \\ &= -y(1-\hat{y}) + (1-y)\hat{y} \\ &= -y + y\hat{y} + \hat{y} - y\hat{y} \\ &= \hat{y} - y\end{aligned}$$

Backpropagation

$$\frac{\partial L}{\partial z} = \hat{y} - y$$

Backpropagation in logistic regression

Goal: $\frac{\partial L}{\partial w}, \frac{\partial L}{\partial b}$

Forward pass

Linear combination

$$z = w^T x + b$$

Activation function **(sigmoid)**

$$\hat{y} = \sigma(z) \qquad \sigma(z) = \frac{1}{1 + e^{-z}}$$

Loss function (binary cross entropy)

$$L(y, \hat{y}) = -\left[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})\right]$$

Computational graph

$$x \rightarrow z \rightarrow \hat{y} \rightarrow L$$

Backpropagation

Now, we calculate the derivative of loss wrt z

$\frac{\partial L}{\partial z}$

Apply the chain rule

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z}$$

Substitute the expressions

$$\begin{aligned} \frac{\partial L}{\partial z} &= \left(-\frac{y}{\hat{y}} + \frac{1 - y}{1 - \hat{y}}\right) \hat{y}(1 - \hat{y}) \\ &= -y(1 - \hat{y}) + (1 - y)\hat{y} \\ &= -y + y\hat{y} + \hat{y} - y\hat{y} \\ &= \hat{y} - y \end{aligned}$$

Backpropagation

$$\frac{\partial L}{\partial z} = \hat{y} - y$$

Backpropagation in logistic regression

Goal: $\frac{\partial L}{\partial w}, \frac{\partial L}{\partial b}$

Forward pass

Linear combination

$$z = w^T x + b$$

Activation function **(sigmoid)**

$$\hat{y} = \sigma(z) \qquad \sigma(z) = \frac{1}{1 + e^{-z}}$$

Loss function (binary cross entropy)

$$L(y, \hat{y}) = -\left[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})\right]$$

Computational graph

$$x \rightarrow z \rightarrow \hat{y} \rightarrow L$$

Backpropagation

Now, we calculate the gradient wrt weights

First, derivative of z wrt the weights

$$\frac{\partial z}{\partial w} = x$$

Now apply the chain rule

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} \frac{\partial z}{\partial w}$$

Replace

$$\frac{\partial L}{\partial w} = (\hat{y} - y)x$$

Backpropagation

$$\frac{\partial L}{\partial z} = \hat{y} - y$$

Backpropagation in logistic regression

Goal: $\frac{\partial L}{\partial w}, \frac{\partial L}{\partial b}$

Forward pass

Linear combination

$$z = w^T x + b$$

Activation function **(sigmoid)**

$$\hat{y} = \sigma(z) \qquad \sigma(z) = \frac{1}{1 + e^{-z}}$$

Loss function (binary cross entropy)

$$L(y, \hat{y}) = -\left[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})\right]$$

Computational graph

$$x \rightarrow z \rightarrow \hat{y} \rightarrow L$$

Backpropagation

Now, we calculate the gradient wrt bias term

First, derivative of z wrt the bias

$$\frac{\partial z}{\partial b} = 1$$

Now apply the chain rule

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial z} \frac{\partial z}{\partial b}$$

Replace

$$\frac{\partial L}{\partial b} = \hat{y} - y$$

Backpropagation

$$\frac{\partial L}{\partial w} = (\hat{y} - y)x$$
$$\frac{\partial L}{\partial b} = \hat{y} - y$$

Backpropagation in logistic regression

Forward pass

Linear combination

$$z = w^T x + b$$

Activation function **(sigmoid)**

$$\hat{y} = \sigma(z) \qquad \sigma(z) = \frac{1}{1 + e^{-z}}$$

Loss function (binary cross entropy)

$$L(y, \hat{y}) = -\left[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})\right]$$

Computational graph

$$x \rightarrow z \rightarrow \hat{y} \rightarrow L$$

Backpropagation

Finally, update weights and bias (gradient descent)

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

Learning rate

Replace gradient:

$$w \leftarrow w - \eta(\hat{y} - y)x$$

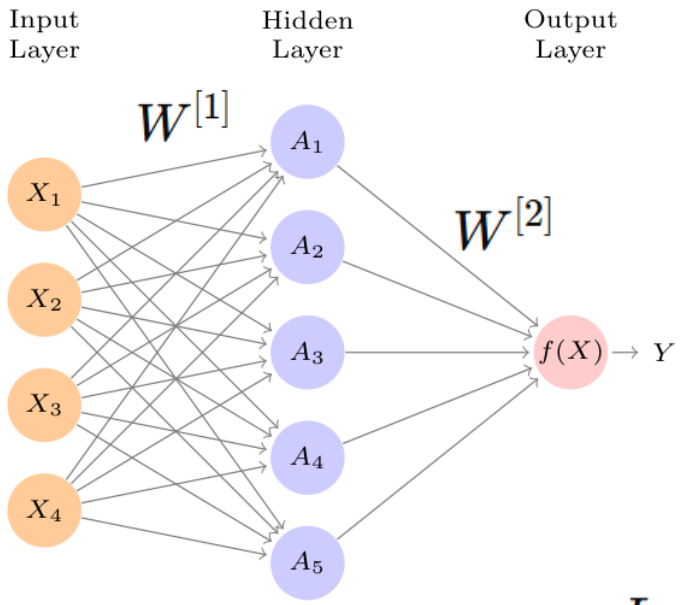
Bias

$$b \leftarrow b - \eta(\hat{y} - y)$$

Backpropagation

$$\frac{\partial L}{\partial z} = \hat{y} - y \qquad \frac{\partial L}{\partial w} = (\hat{y} - y)x \qquad \frac{\partial L}{\partial b} = \hat{y} - y$$

Forward propagation



Backpropagation

Forward propagation

Hidden layer

$$z^{[1]} = W^{[1]}x + b^{[1]}$$

$$a^{[1]} = g(z^{[1]})$$

Output layer

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

$$\hat{y} = a^{[2]} = \sigma(z^{[2]})$$

Loss function

$$L = -\left[y\log(\hat{y}) + (1-y)\log(1-\hat{y})\right]$$

Backpropagation

Output layer

$$dz^{[2]} = \frac{\partial L}{\partial z^{[2]}} = a^{[2]} - y$$

$$dW^{[2]} = dz^{[2]}(a^{[1]})^T$$

$$db^{[2]} = dz^{[2]}$$

Hidden layer

$$da^{[1]} = (W^{[2]})^T dz^{[2]}$$

$$dz^{[1]} = da^{[1]} * g'(z^{[1]})$$

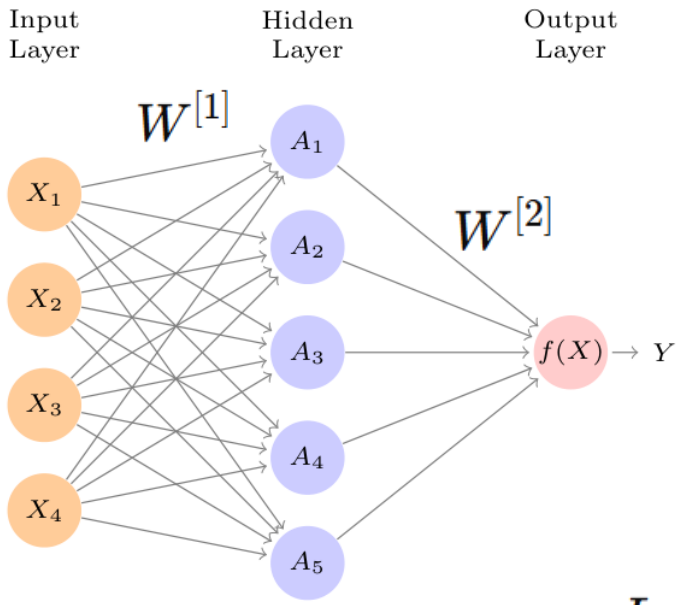
$$= da^{[1]} * a^{[1]}(1 - a^{[1]})$$ **if sigmoid**

$$dW^{[1]} = dz^{[1]}x^T$$

$$db^{[1]} = dz^{[1]}$$

Backpropagation

Forward propagation



Backpropagation

Forward propagation

Hidden layer

$$z^{[1]} = W^{[1]}x + b^{[1]}$$

$$a^{[1]} = g(z^{[1]})$$

Output layer

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

$$\hat{y} = a^{[2]} = \sigma(z^{[2]})$$

Loss function

$$L = -\left[y\log(\hat{y}) + (1-y)\log(1-\hat{y})\right]$$

Backpropagation

Gradient descent updates

$$W^{[2]} \leftarrow W^{[2]} - \eta dW^{[2]}$$

$$b^{[2]} \leftarrow b^{[2]} - \eta db^{[2]}$$

$$dW^{[2]} = dz^{[2]}(a^{[1]})^T$$

$$db^{[2]} = dz^{[2]}$$

Hidden layer

$$W^{[1]} \leftarrow W^{[1]} - \eta dW^{[1]}$$

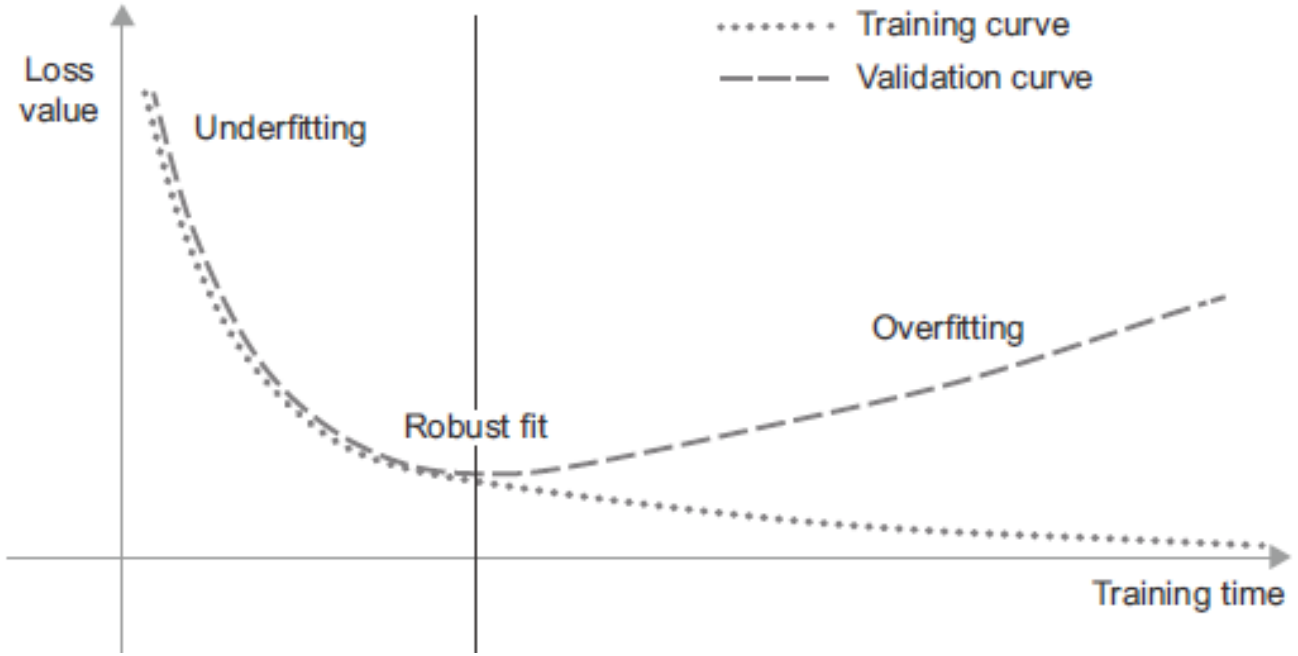
$$b^{[1]} \leftarrow b^{[1]} - \eta db^{[1]}$$

$$dW^{[1]} = dz^{[1]}x^T$$

$$db^{[1]} = dz^{[1]}$$

Training for generalisation

- Due to overparameterisation, NNs will always overfit eventually
  - Eventually, it will memorise training data!
- It is common to monitor validation loss & stop when it starts increasing



Training for generalisation

Regularisation

- When building a NN, it is common to build a network that overfits
- Once our model overfits, we can apply regularisation techniques
  - Reducing model size
  - Weight regularisation
  - Dropout

Training for generalisation

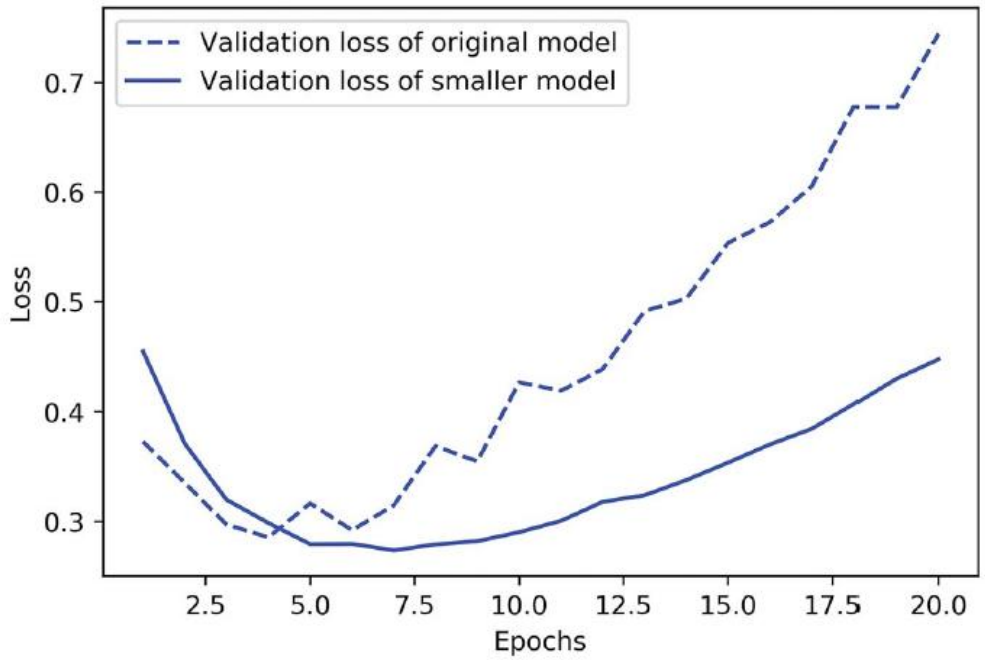
Regularisation: reducing network size

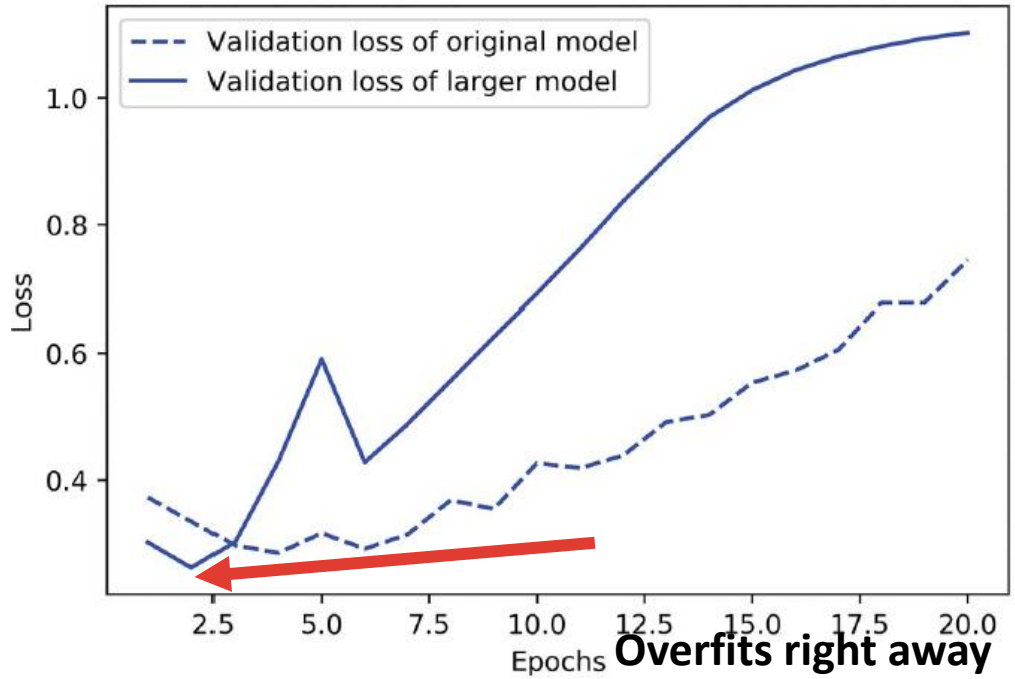
- Make model too small to memorise training data
- Instead, it will try to memorise a compressed representation √
- …but make sure the model has enough parameters!
- We need to find a compromise, but there is no magic formula
- We must evaluate an array of different architectures
- Start small and increase size and # of layers

Training for generalisation

Regularisation: reducing network size (example)

- Layer size: 16 (original), 4 (smaller) and 512 (larger)





Overfits right away

Training for generalisation

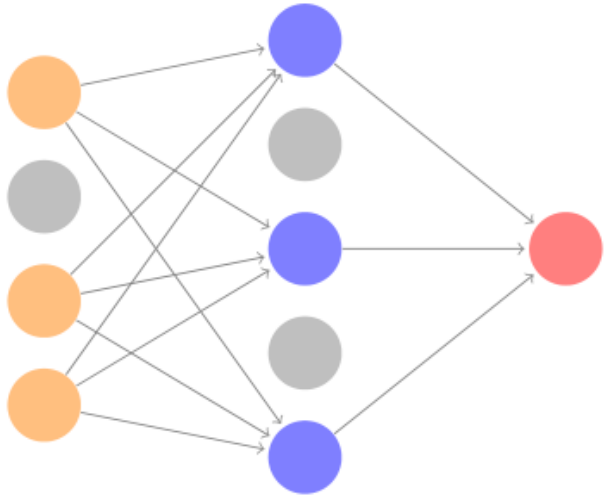
Regularisation: weight regularisation

- Force model weights to take small values
- How? We add a cost to the loss function
- Two main flavours:
  - L1 regularisation: proportional to absolute value of weights
  - L2 regularisation (weight decay): proportional to square of weights
- Cost only added at training time
- It works in smaller models, not so much in big ones

Training for generalisation

Regularisation: Dropout

- Dropping out (setting to zero) random units during training
- Dropout rate: fraction of features zeroed out (between 0.2 and 0.5)



0.3	0.2	1.5	0.0
0.6	0.1	0.0	0.3
0.2	1.9	0.3	1.2
0.7	0.5	1.0	0.0

50% dropout →

0.0	0.2	1.5	0.0
0.6	0.1	0.0	0.3
0.0	1.9	0.3	0.0
0.7	0.0	0.0	0.0

* 2

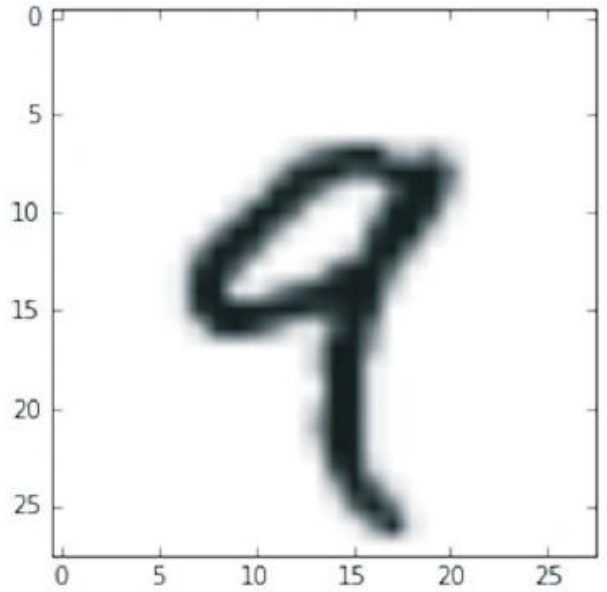
Building neural networks

Tensors

- Deep learning consists of chains of tensor operations
- **Tensor** = multidimensional array
- Types of tensor by rank
  - Scalar: rank-0 tensor
  - Vector: rank-1 tensor
  - Matrices: rank-2 tensor: tabular data
  - Rank-3 or higher: colour images, video, time series data
- Tensors can contain integers or floats

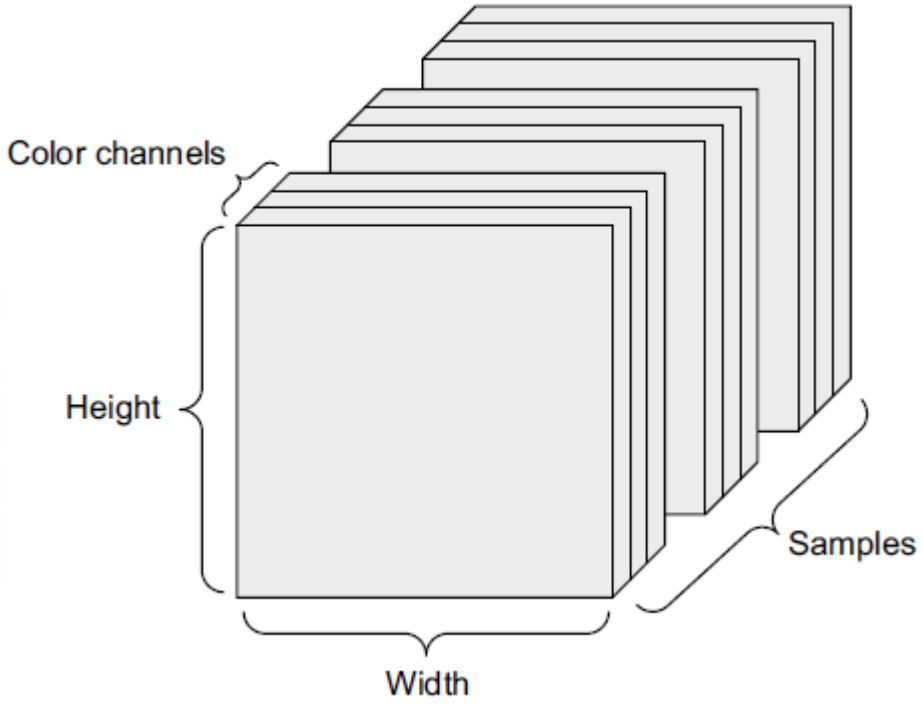
Tensors

Examples of tensors



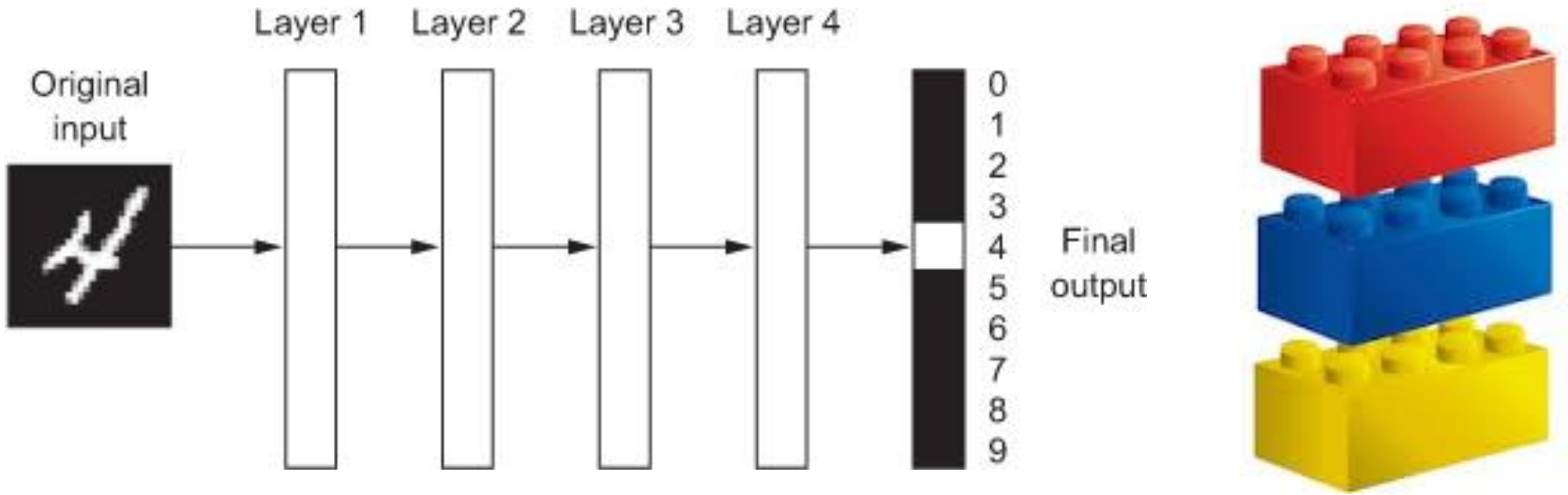
Rank-2

Rank-3



Rank-4

Building neural networks

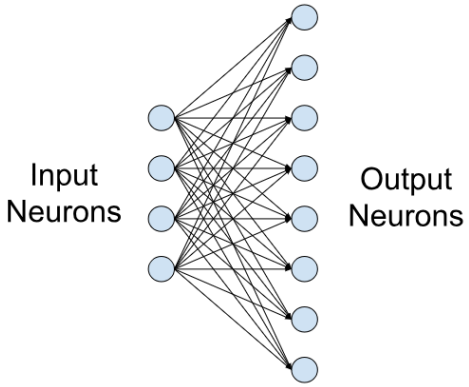


- Layer are the fundamental data structure of neural networks
- Layer: input tensor → output tensor
- State: weights

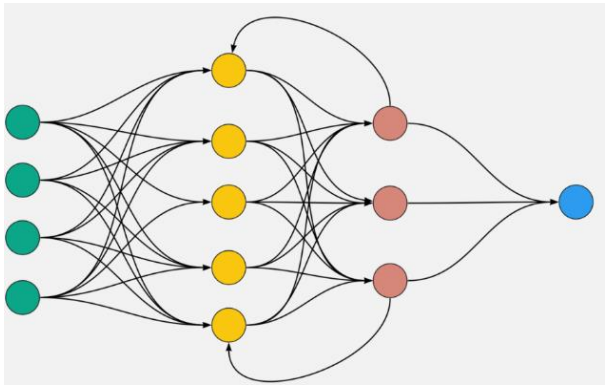
Layers: The building blocks of deep learning

Different layers appropriate for different inputs / tensors

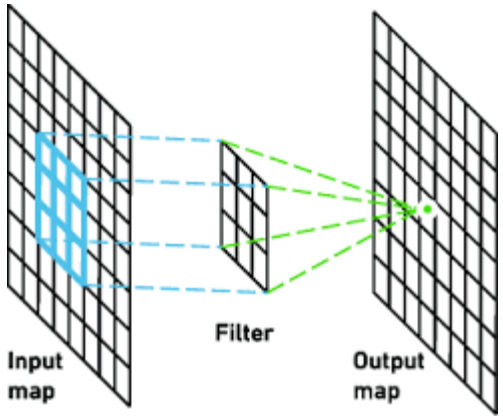
- Rank-2: Vector (samples, features): fully connected or dense
- Rank-3: Sequence (samples, timesteps, features): Recurrent
- Rank-4: Image (samples, height, width, channels,): Convolutional



Dense layer



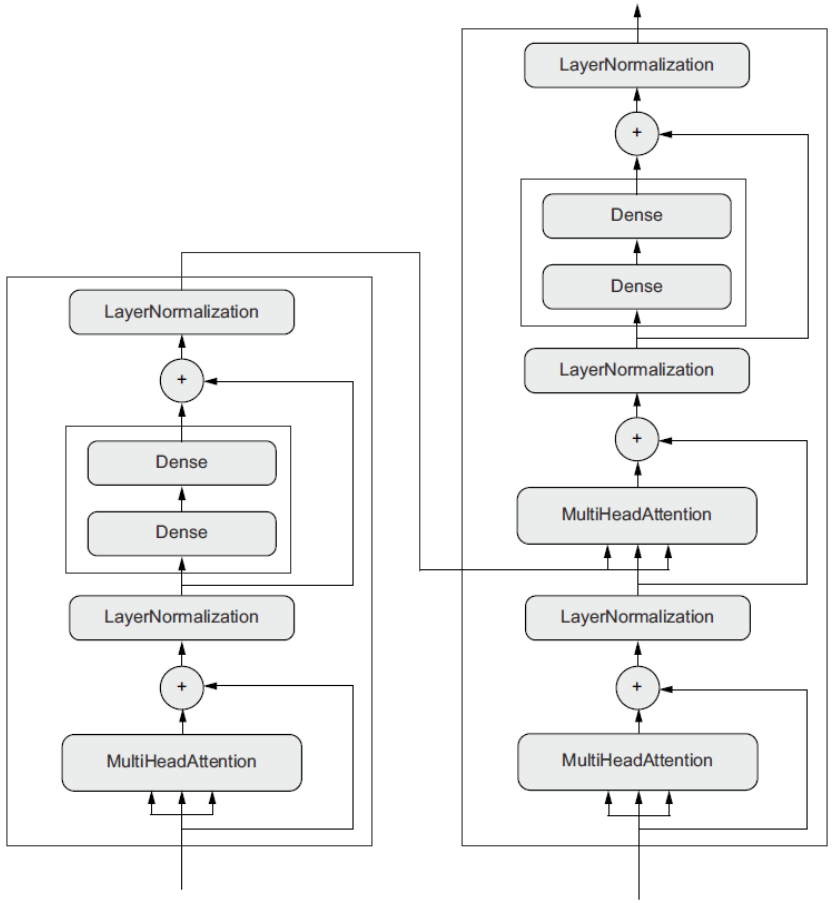
Recurrent layer



Convolutional layer

Selecting the right architecture

- Deep learning model = graph of layers
- So far, we've seen sequential stacks of layers
- But more complex networks common
- Topology defines *hypothesis space*
- Principles can guide architecture…
- …but only practice will get you there



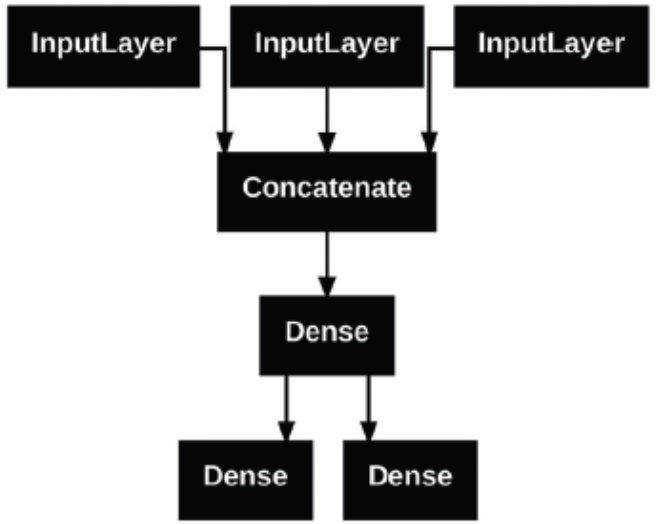
Transformer architecture

Multimodal inputs, multiple outputs

- Neural networks can accommodate multimodal inputs and multiple outputs

Example

- Costumer support tickets for a computer shop.
- **Input**: title, text, tags (categorical, one-hot encoded)
- **Output**: priority score between 0 and 1, department assigned (IT, sales, …)



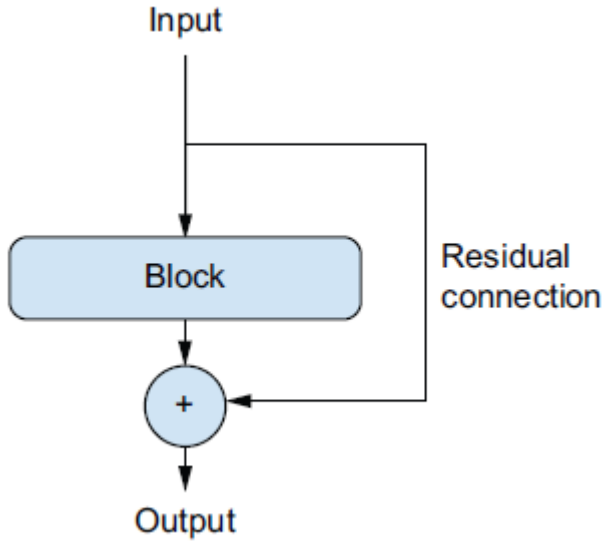
Vanishing gradient problem

- Deep stack of narrow layers outperforms shallow stack of large layers
- But:
  - There's a limit to depth due to *vanishing gradients*
  - In backpropagation, we use the chain rule to propagate gradients
  - But this leads to a chain of multiplications
  - If one or more of multiplied gradients is near zero, the result is near zero
  - In that case, weights are barely updated
  - The deeper the network, the more likely a gradient is near zero

Residual connections

- Deep neural networks face two main issues:
  - The vanishing gradient problem
  - Each layer introduces noise: broken phone game

- *Residual connections* alleviate these issues
  - Connections that skip layers
  - Retain a noiseless version of information

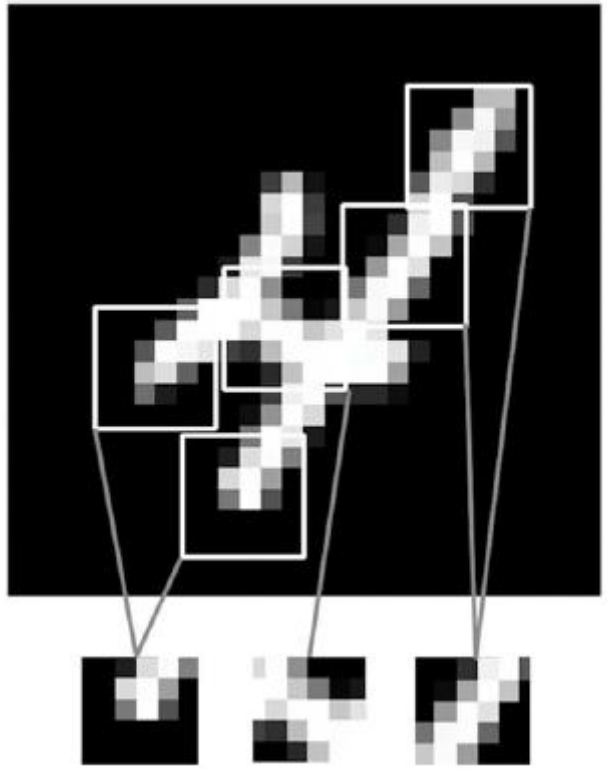


Batch normalisation

- Normalisation makes samples more similar to each other
- Most common: centre the data around zero and divide by SD
- Usually done before feeding it to models
- …but why not do it after every layer?
- *Batch normalisation* is a layer that does just that using current batch
- Unclear why, but helps with gradient propagation, allows deeper networks

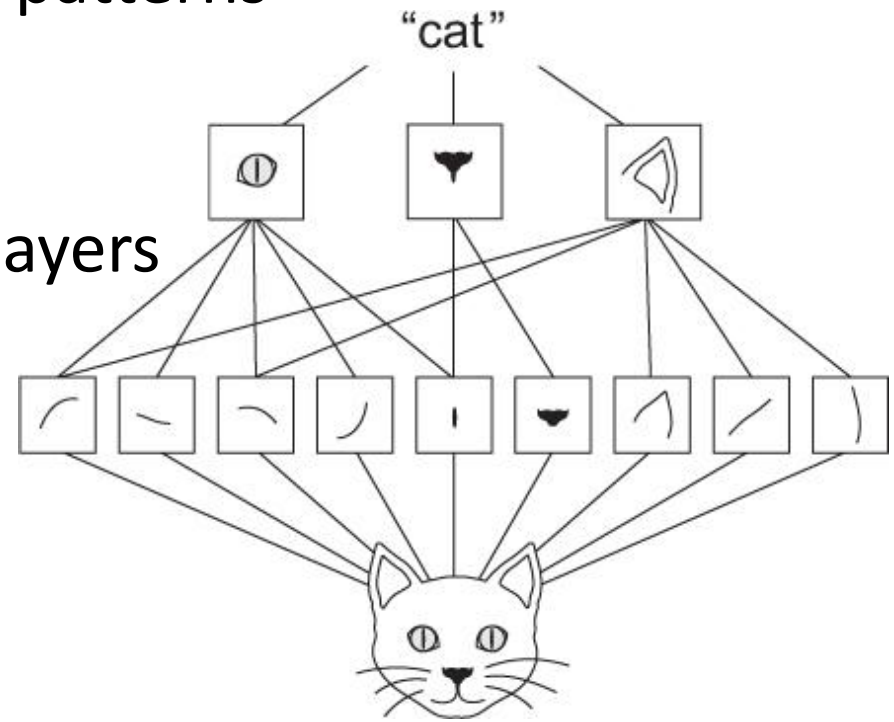
Convolutional neural networks

Introduction to convolutional neural networks



The convolution operation

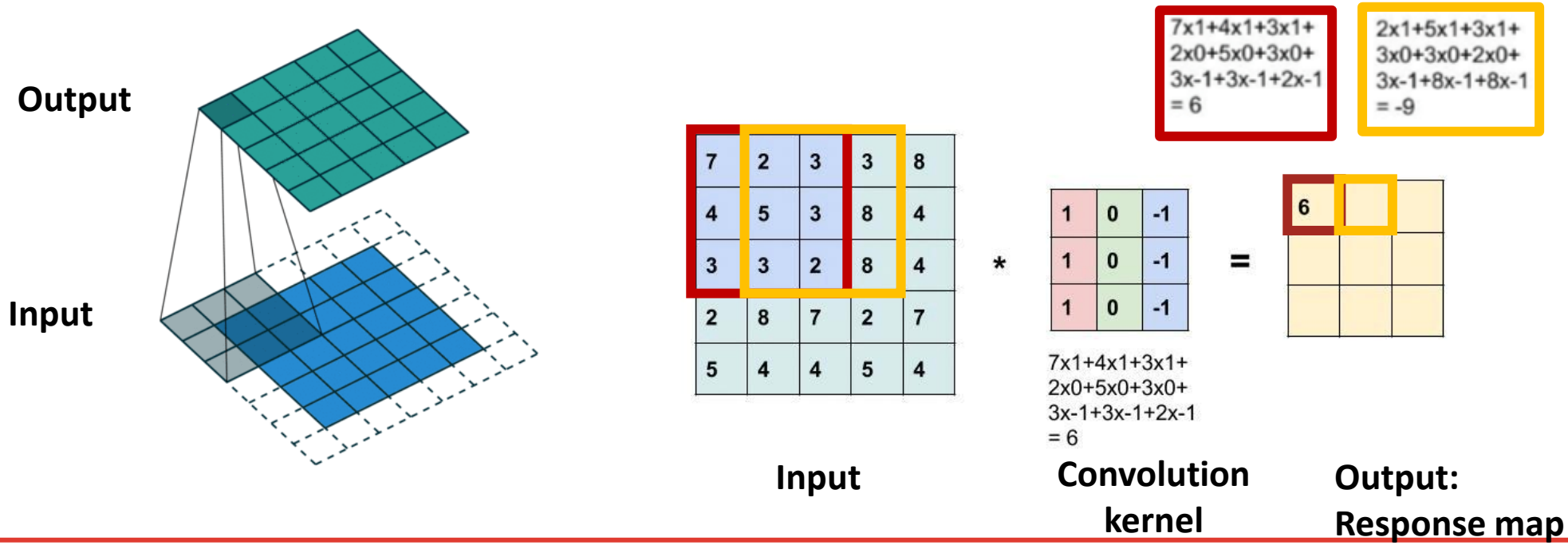
- Dense layers learn <u>global</u> patterns
- Convolution layers learn <u>local</u> patterns
  - edges, textures
  - translation invariant
- A sequence of convolutional layers can learn a spatial hierarchy



Introduction to convolutional neural networks

The convolution operation

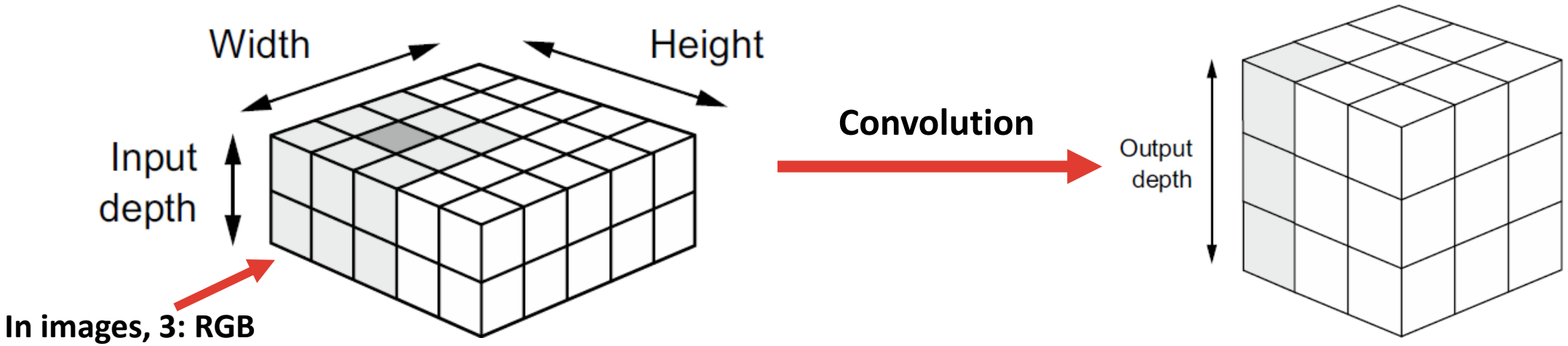
- Slide window over patches of input, calculate dot product with kernel
- *Convolution kernel*: (learned) weight matrix



Introduction to convolutional neural networks

The convolution operation

- Convolutions operate over rank-3 tensors called input feature maps
- Convolution applied to input map → output feature map
- Output feature map still a rank-3 tensor with different depth
- Output depth depends on the number of *filters*



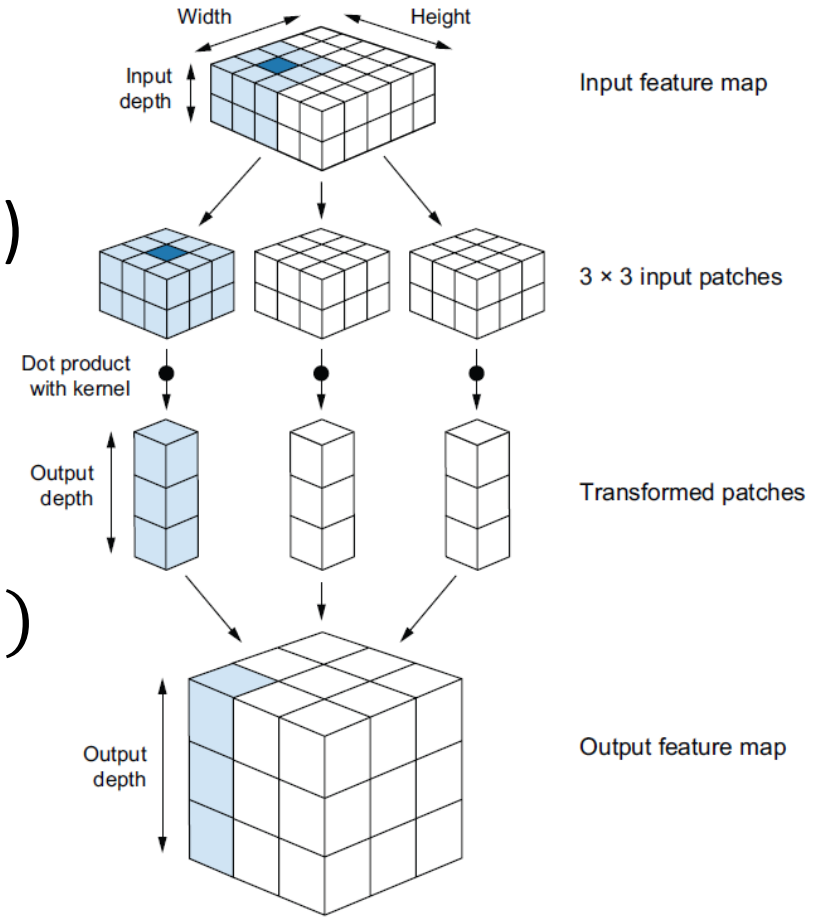
Introduction to convolutional neural networks

The convolution operation

It works sliding windows over **3D** input feature map

- Input: (kernel_height, kernel_width, input_depth)
- Output: (output_depth,)
- $output_i = \sum_{j=1}^{input\_depth} dot(input, kernel_{i,j})$
- Number of weights =

$$(kernel\_height\ x\ kernel\_width\ +\ 1)\ x\ input\_depth\ x\ output\_depth$$

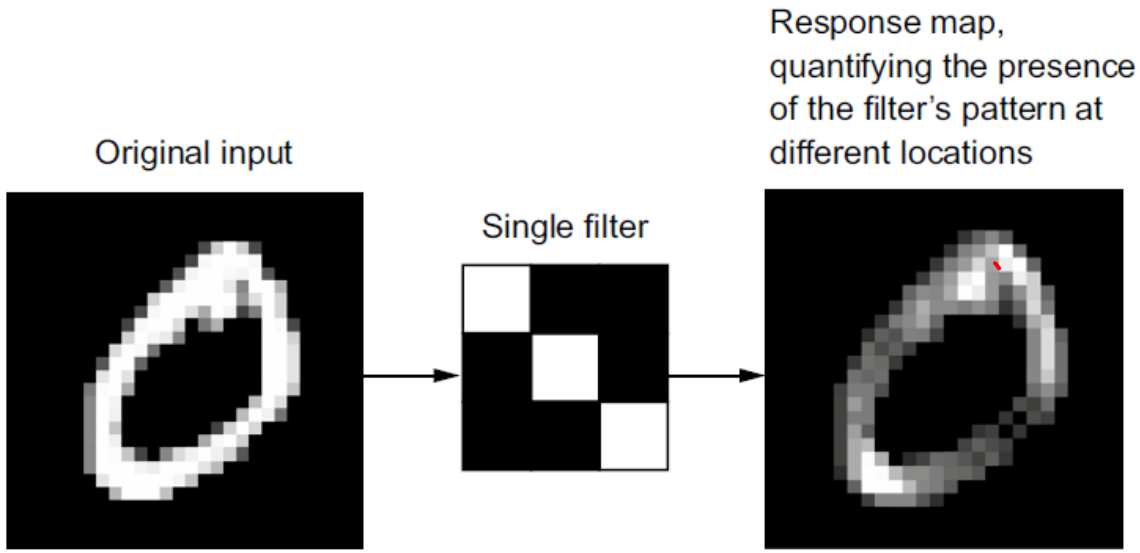


Introduction to convolutional neural networks

The convolution operation

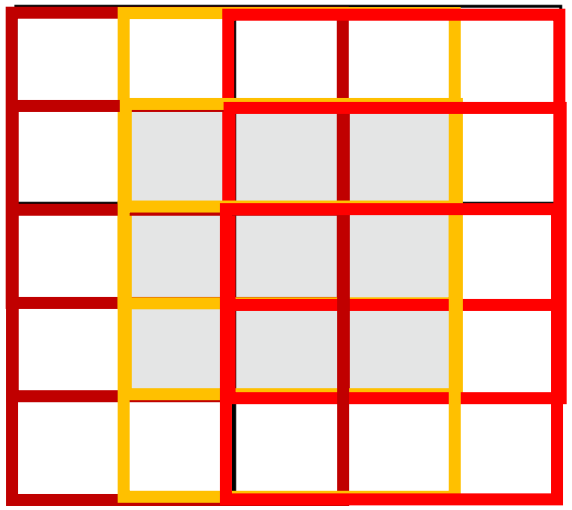
Filters

- Encode aspects of input data (e.g. presence of face, diagonal line)
- Filter ≈ set of convolution kernels (one per input channel)

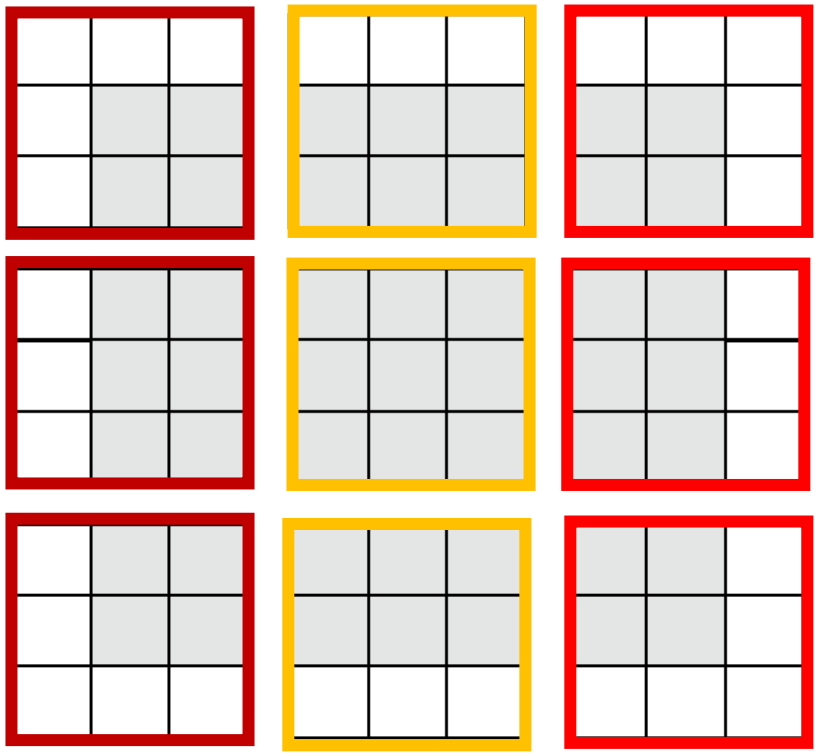


The convolution operation

Understanding border effects and padding



5x5 input feature map



Valid locations of 3x3 patches

Only 9 tiles

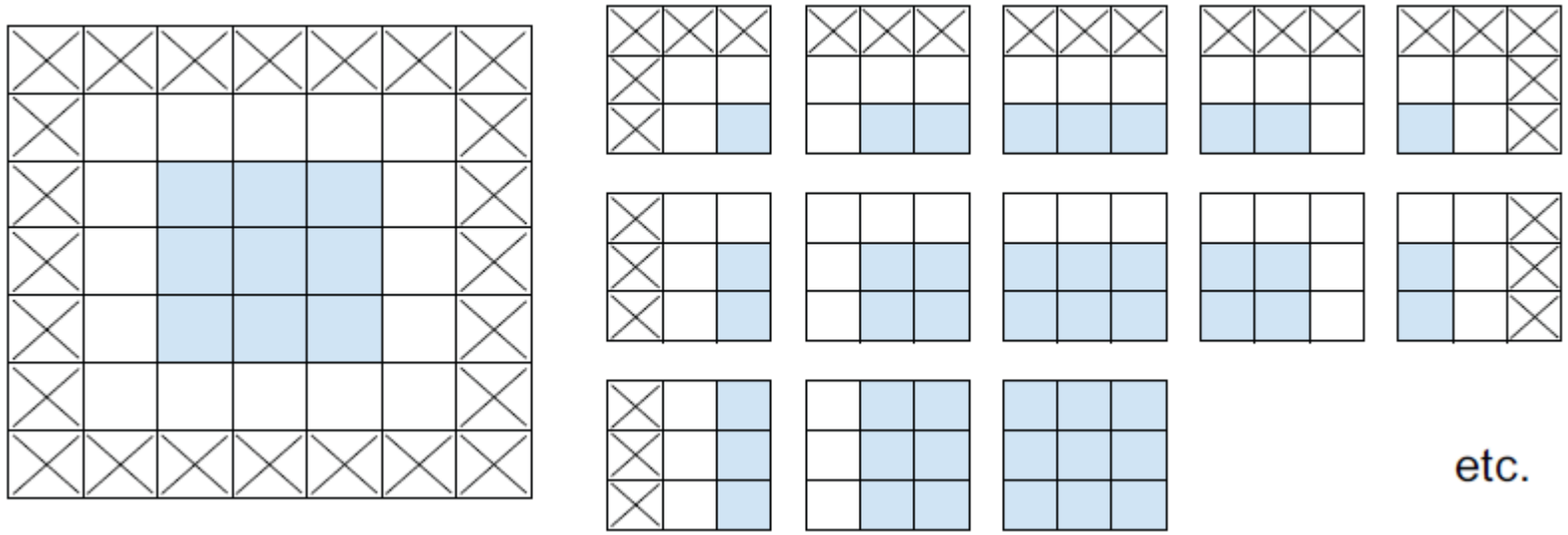
Output feature map size: (3x3)

Border effect!

The convolution operation

Understanding border effects and padding

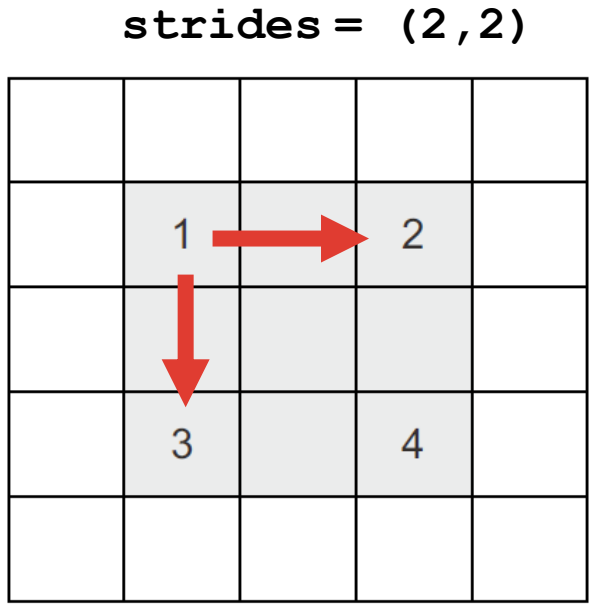
- To keep size, we use *padding*: add rows and columns
- In Keras, `padding` argument '`valid`' (none) or '`same`' (same output size)



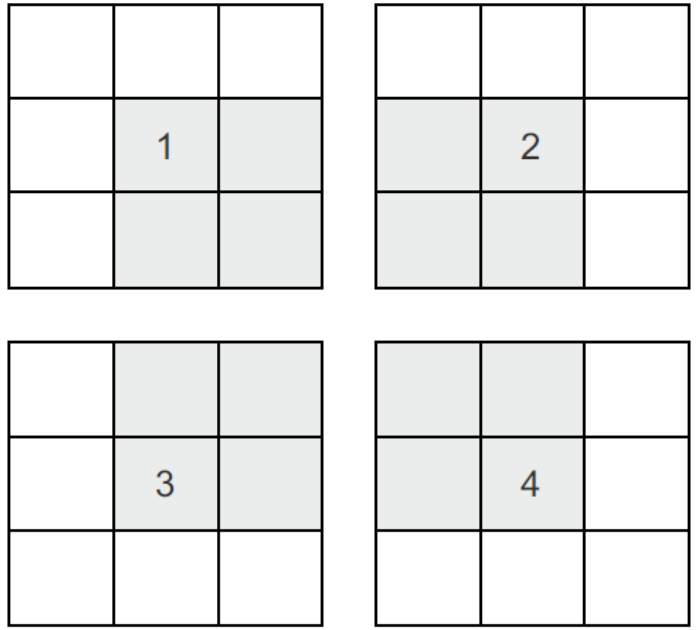
The convolution operation

Understanding strides

- Stride is another parameter: distance between two windows



5x5 input feature map



Output feature map size: (2x2)

Introduction to convolutional neural networks

The max-pooling operation

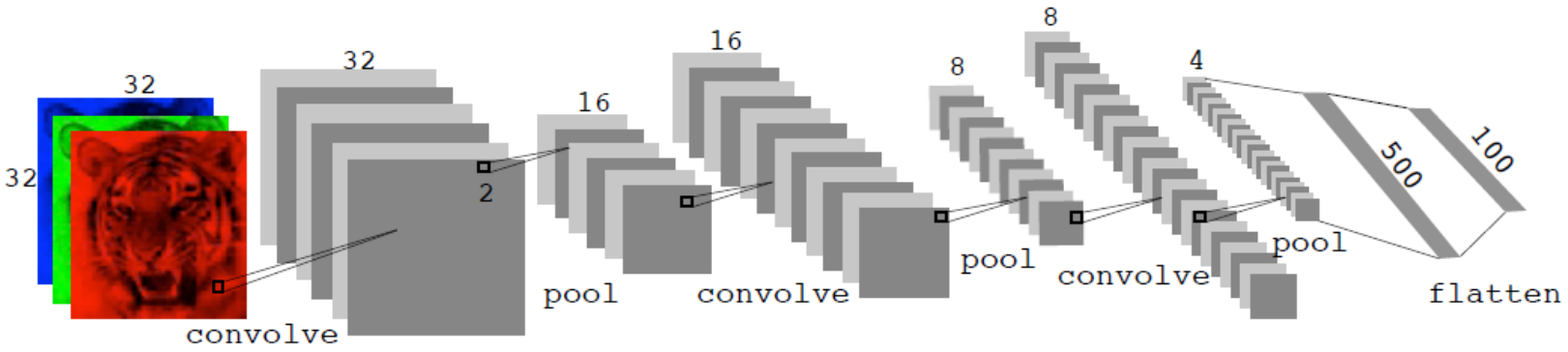
- How does max-pooling work?
  - Extract patches from input and output max value
  - Unlike convolution, not learnable (hardcoded max)
  - Usually 2x2 patches, stride 2

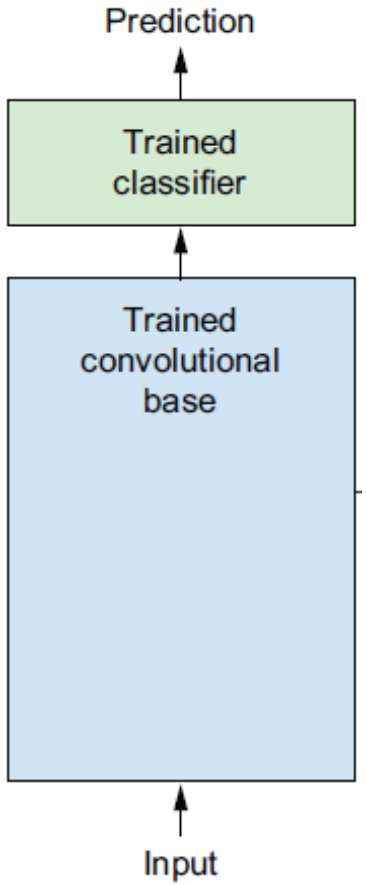
$$\begin{bmatrix} 1 & 2 & 5 & 3 \\ 3 & 0 & 1 & 2 \\ 2 & 1 & 3 & 4 \\ 1 & 1 & 2 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} 3 & 5 \\ 2 & 4 \end{bmatrix}$$

- Why down sample maps?
  - To enable the model to learn a spatial hierarchy of features
  - Deeper conv layers will look at bigger larger of input
  - Reduce parameters: 10^7 to 10^5 that is, $1/100^{th}$ of the size
  - Alternatives to max pooling (strides, avg pooling) not so informative

Introduction to convolutional neural networks

Architecture of a convolutional neural network

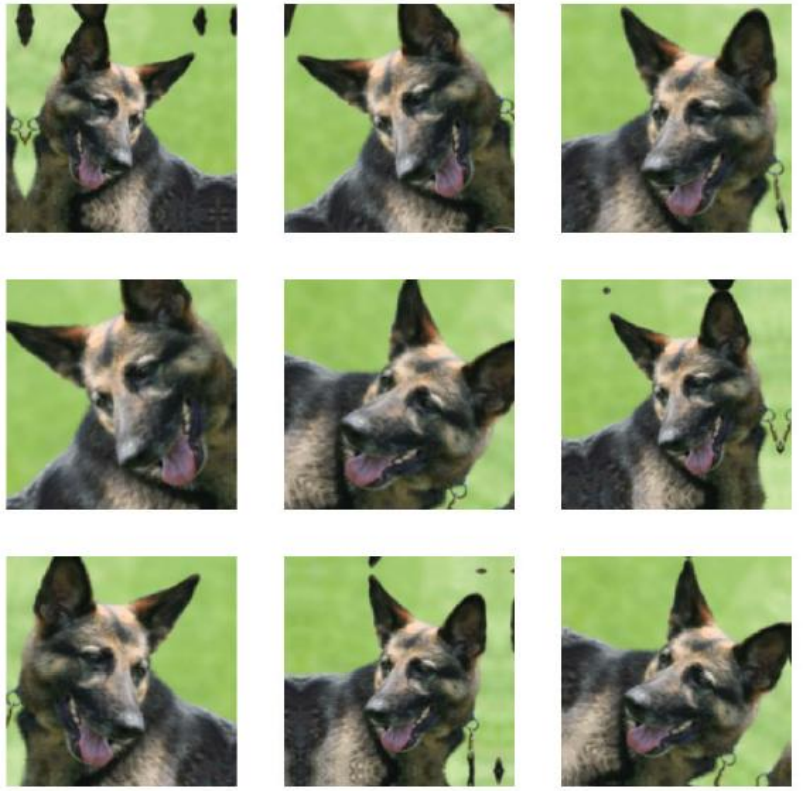




Introduction to convolutional neural networks

Using data augmentation

- Alleviate issue of overfitting due to limited data
- Generate new data applying random transformations to training data. Eg:
  - Rotation
  - Flipping
  - Zoom
- Generated data needs to be ‘believable’
  - i.e., valid samples of distribution



Transfer learning with CNNs

- Deep learning models usually require big datasets to train
- When our dataset is small, we can adapt a model trained on a similar task
- Pretrained model: model previously trained on large dataset
- In vision, features learnt by pretrained model = model of visual world
- These features can be useful in different vision tasks
  - E.g. train model on ImageNet and adapt it to identify furniture
- This portability, called *transfer learning,* is key advantage of deep learning
- Two ways to use pretrained model:
  - Feature extraction
  - Fine-tuning

Transfer learning with CNNs

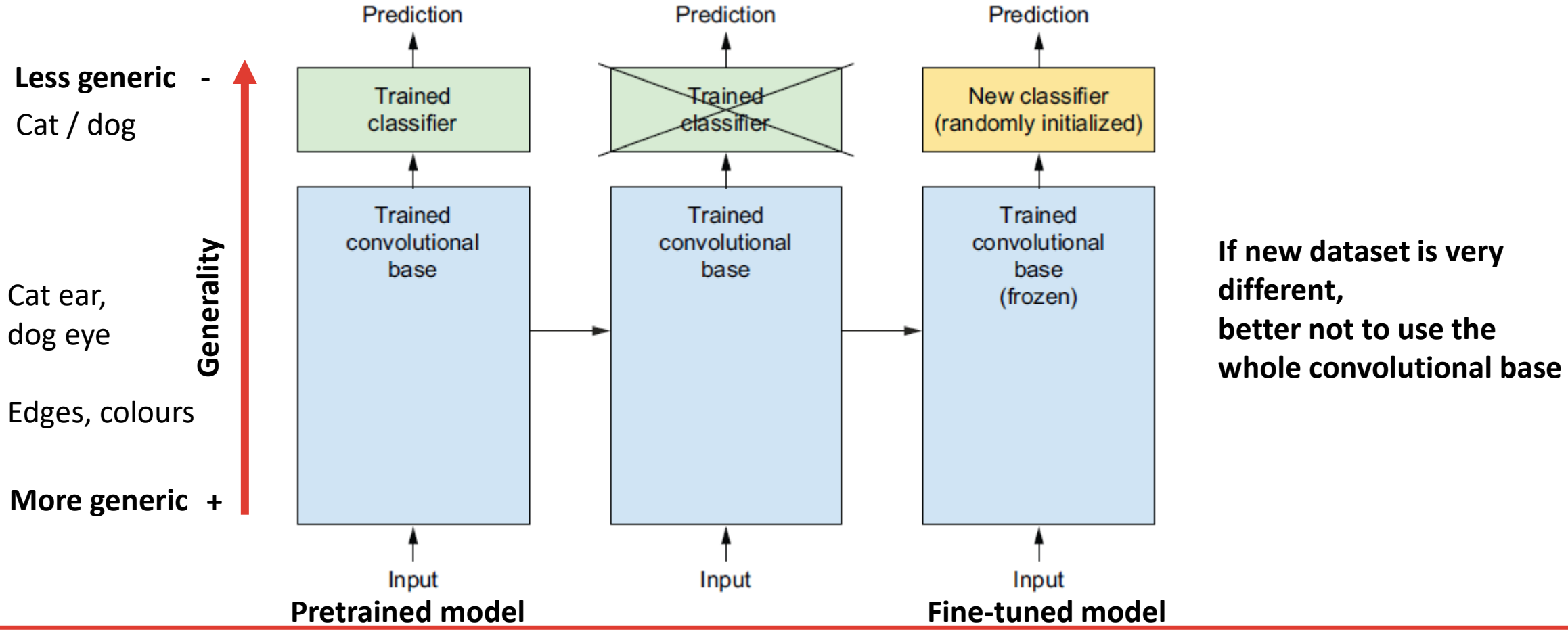
Feature extraction with a pretrained model

Two options:

- **Option 1:** Standalone classifier
  - Run data through convolutional base, use output to build classifier
  - Faster, cheaper
- **Option 2:** Extend the model
  - Add dense layer(s) to the convolutional base
  - Freeze convolutional base and train only the dense layer(s)
  - Slower, more expensive

Transfer learning with CNNs

Feature extraction with a pretrained model



If new dataset is very different,
better not to use the whole convolutional base

Transfer learning with CNNs

Feature extraction with a pretrained model: extend model

Steps:

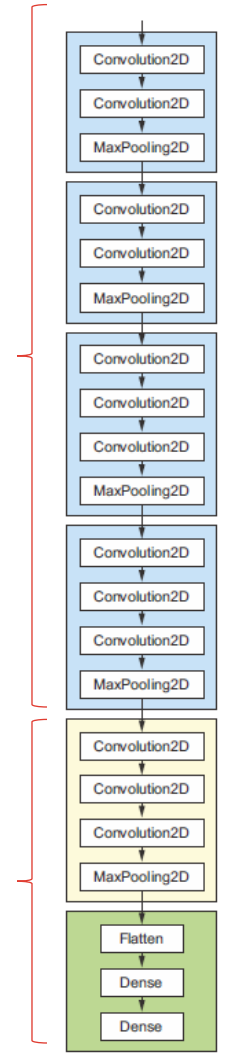
- Freeze convolutional base (do not update weights during training)
- Add dense layer(s) to convolutional base (and data augmentation)
- Train only the dense layer(s)

Transfer learning with CNNs

Fine-tuning a pretrained model

- Fine-tuning: unfreeze top layers and train with new layers
- Steps
  1. Add new layers on top of pretrained model
  2. Freeze pretrained model
  3. Train new layers
  4. Unfreeze a few layers of pretrained model
  5. Jointly train these layers with new layers

Same as in feature extraction (option 2)



Frozen

Conv base

Fine-tune

New

Transfer learning with CNNs

Fine-tuning a pretrained model

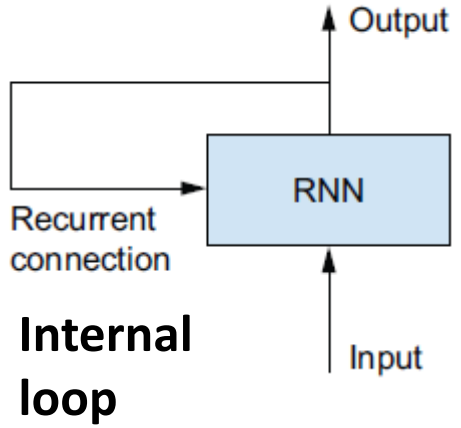
Unfreezing a few layers

- We usually we unfreeze only the last few conv layers
- Why not more?
  - Earlier layers encode more generic, reusable features
  - Retraining these will probably not improve our model
  - The more parameters you train, the more you risk overfitting

Recurrent neural networks

Understanding recurrent neural networks

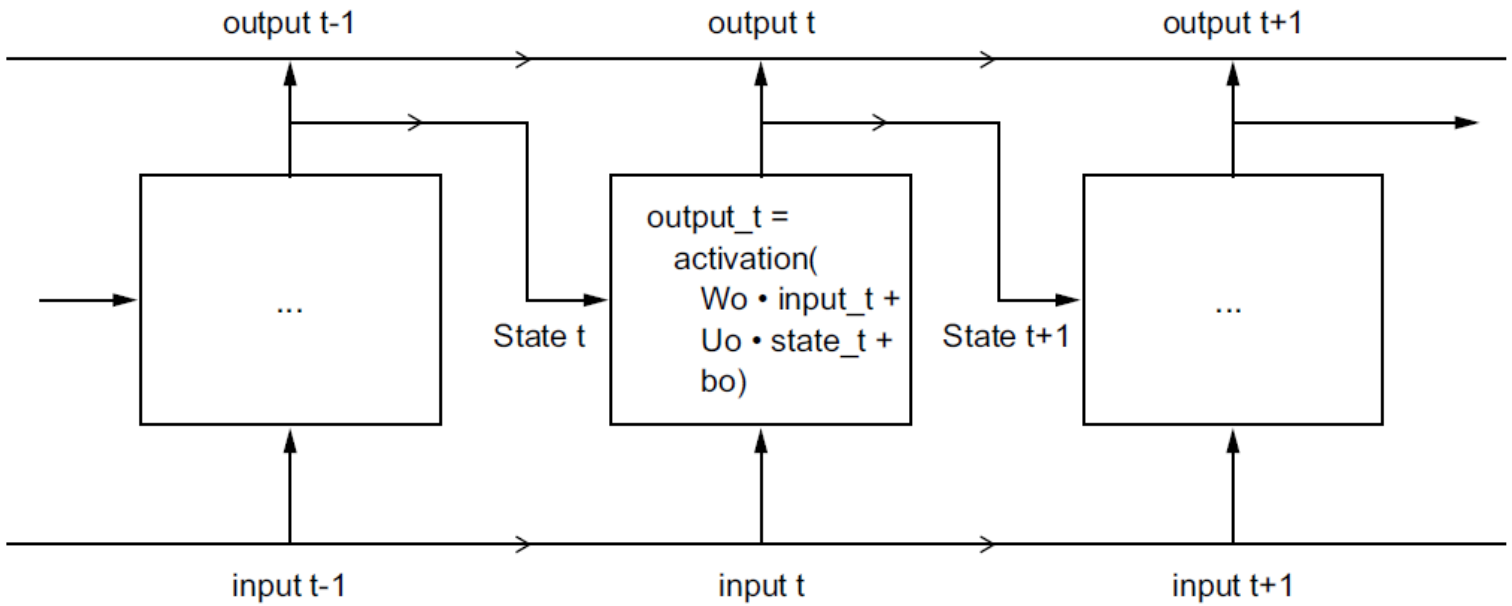
- So far, all NNs we've seen have no memory
- Each input processed independently, with no 'state' kept between inputs
- Such networks are called *feed-forward networks*
- Contrarily, when we read a sentence, we ingest information word by word
- Incrementally updating an 'internal state'
- Recurrent NNs (RNNs) adopt this principle
  - Process sequences iterating through elements
  - Maintaining a *state* with info seen so far
  - State is reset between processing two sequences
  - One sequence is a single data point



Understanding recurrent neural networks

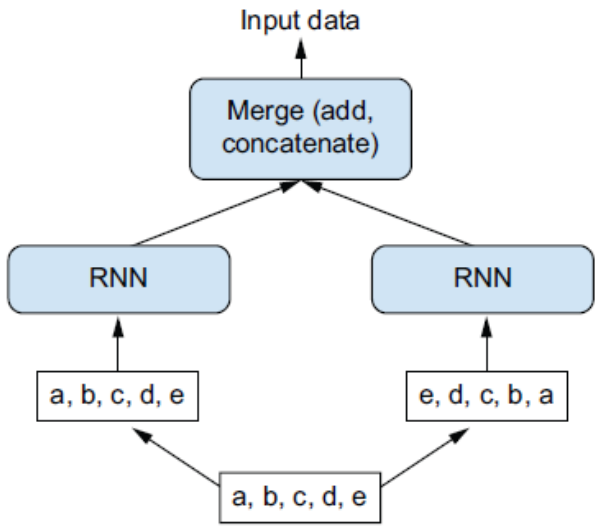
RNNs are defined by their *step function*, e.g.:

$$output_t = tanh(W * input_t + U * state\_t + b)$$



Bidirectional RNNs

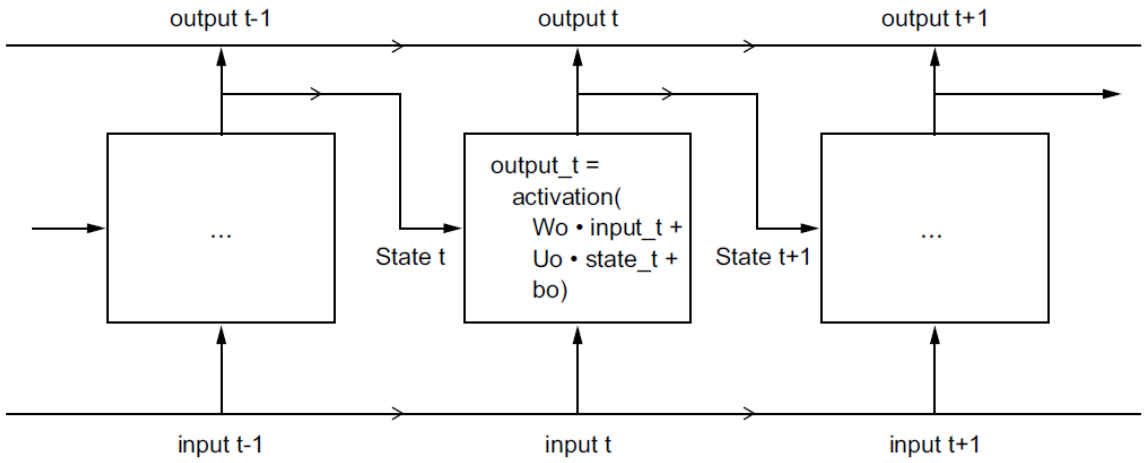
- They can work better in certain tasks (e.g., NLP)
- RNNs are order-dependent:
  - Reversing it will change learnt representations
- Bidirectional RNNs:
  - Use two RNNs
  - Each RNN processes input in one direction
  - Then merge them
  - They can identify patterns missed by unidirectional RNNs



Limitations of simple RNNs

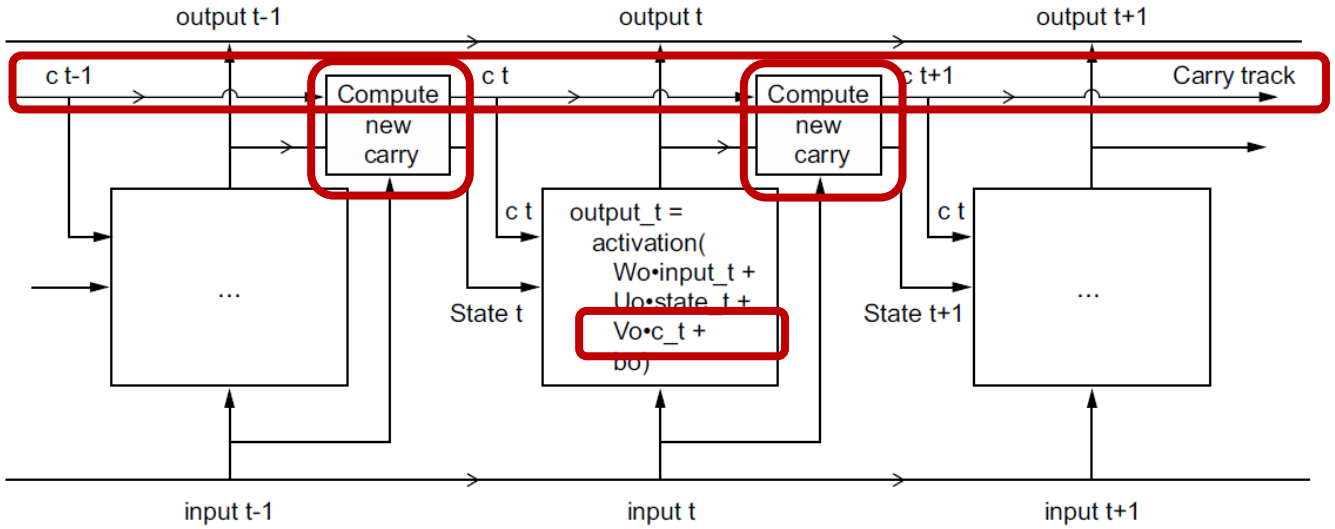
- Simple RNNs have a fatal limitation: they forget too easily
- They forget what they saw many timesteps before
- This means they cannot handle long sequences (e.g. long sentences)
- This due to the *vanishing gradient problem*
- This issue is solved with something similar to residual connections

Long short-term memory (LSTM)



Simple RNN

Residual connections ≈ Conveyor belt



LSTM

How do we calculate the new carry?

Long short-term memory (LSTM)

How do we calculate the new carry?

In brief:

$$c_t = old\ memory + new\ memory$$

More specifically, =four elements:

- Previous cell state c_{t-1}
- The forget gate f_t [0-1]: decides how much all memory to keep
- The input gate i_t: decides how much new information to write
- The candidate memory $\tilde{c}_{t-1}$ creates possible new content

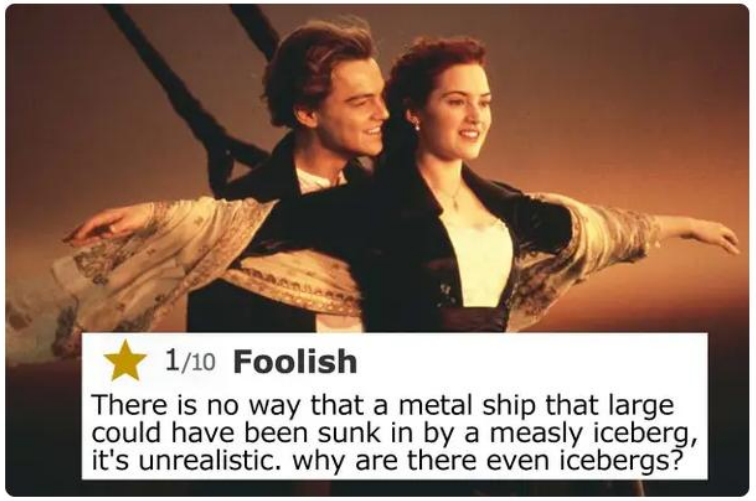
$$c_t = f_t * c_{t-1} + i_t * \tilde{c}_{t-1}$$

Example: Text classification

Example: text classification

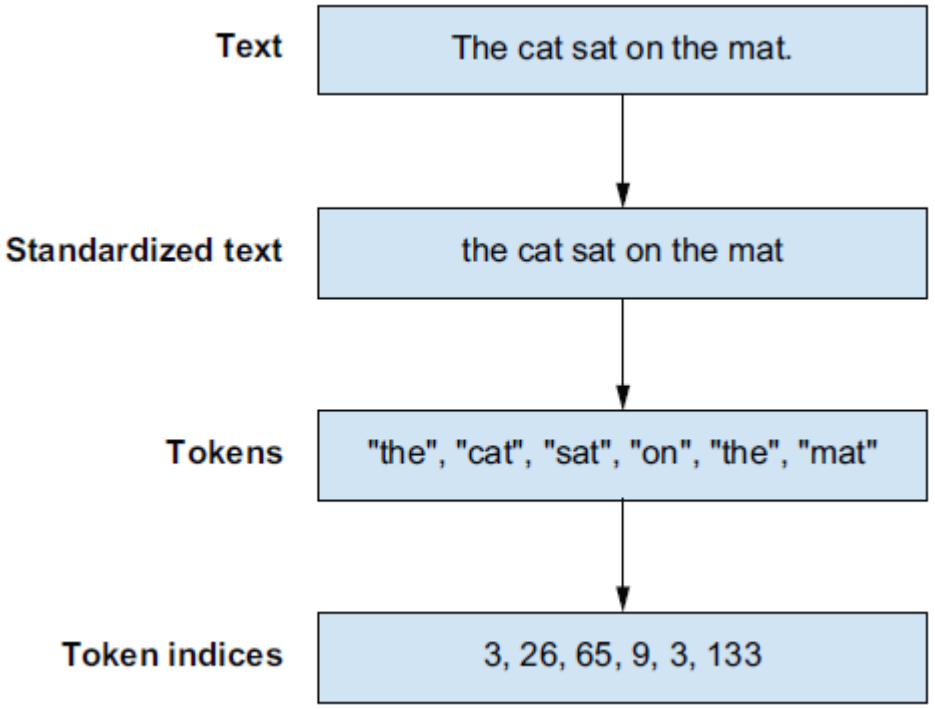
The IMDB dataset

- Aim: classify reviews as positive or negative based on text
- 50,000 highly polarised reviews, split into:
  - 25,000 training
  - 25,000 testing
- Each with 50% positive and negative reviews



Preparing text data

- DL models cannot receive text as inputs
- *Vectorising*: transforming text to numeric tensors
  - Standardise text: e.g. convert to lowercase, remove punctuation.
  - Tokenisation: Split text into units (*tokens,* e.g. words or characters)
  - Indexing converting each token to indices using a vocabulary



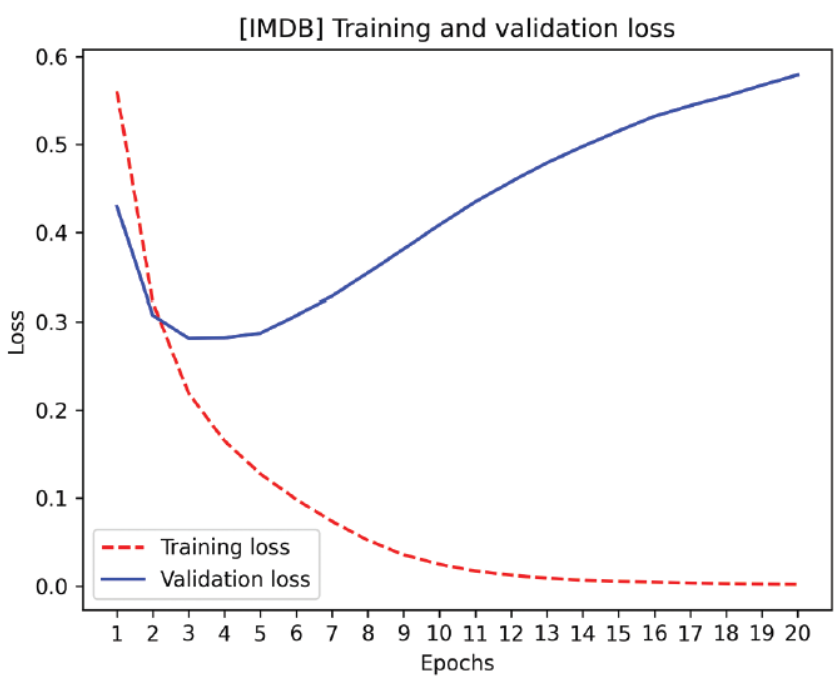
Preparing the data

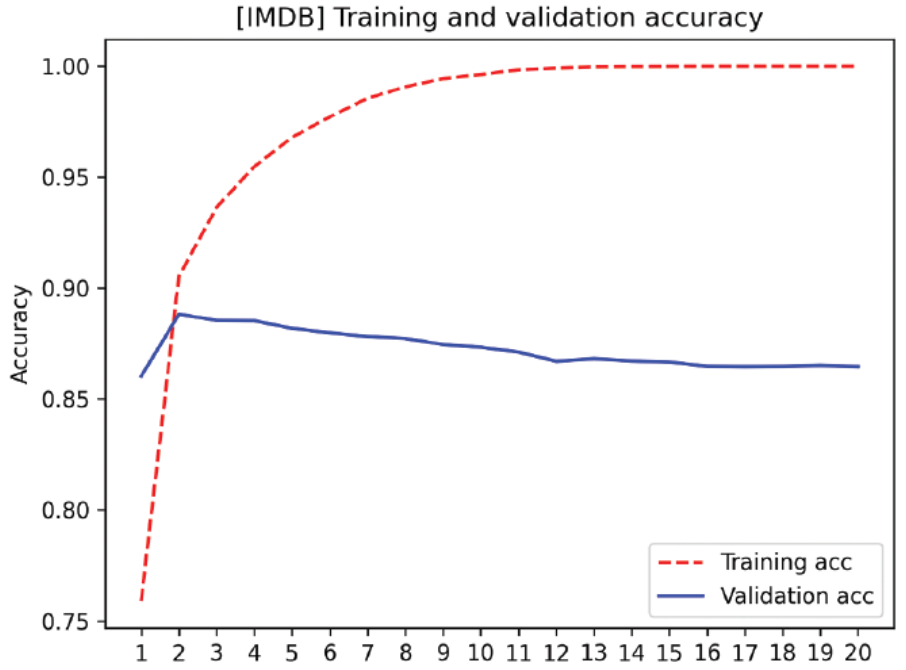
- We can't feed the NN lists of integers of varying length
- How to fix it:
  - Solution 1: Sequence modelling
    - Pad & truncate integer lists to `max_length`
    - *one-hot encode* each integer: Ex.: 4 → [0,0,0,,1,0,0,…,0] (vocab_size)
    - turn into rank-3 tensor `(samples,vocab_size,max_length)`
    - use an *embedding* input layer
  - Solution 2: Bag of words
    - *Multi-hot encoding* into 0s and 1s. Ex.: [4,3] → [0,0,0,1,1,0,0,…,0] (vocab_size)
    - turn into rank-2 tensor `(samples, vocab_size)`
    - `Dense` input layer

Bag-of-words approach

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 20000)	0
dense (Dense)	(None, 1)	20,001

Test accuracy ~88%



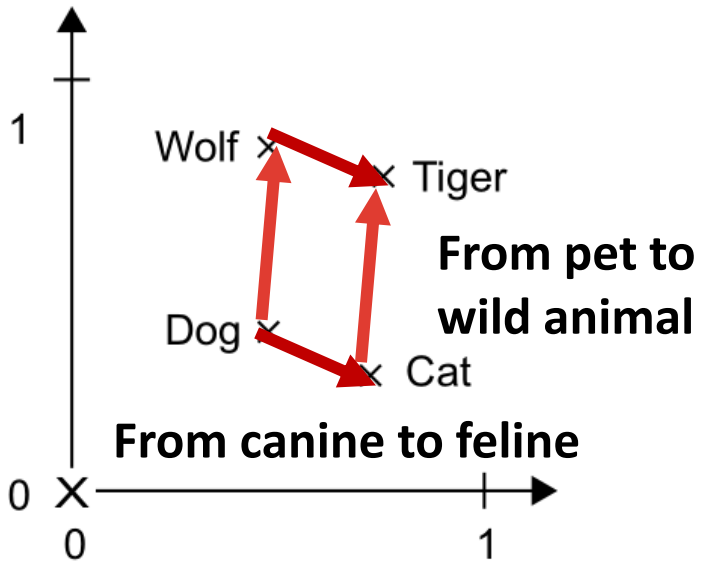


Understanding word embeddings

- One-hot encoding implies all tokens are independent of each other
- But words form a structured space, they share information:
  - E.g. words ‘film’ and ‘movie’ are almost interchangeable, but vectors representing them are orthogonal to each other
  - They should be the same vector, or close enough
- A good representation should reflect this:
  - Synonyms would have similar representations, and
  - *Geometric distance* should relate to *semantic distance*
- Word embeddings are vector representations that achieve this

Understanding word embeddings

- Word embeddings are structured representations
- Specific directions in embedding space are meaningful



Common examples of meaningful geometric transformations:

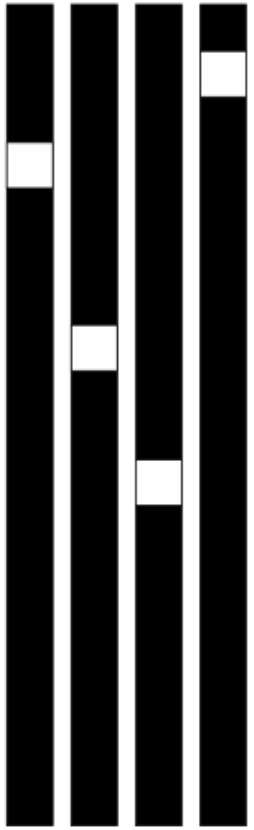
- **Gender (e.g., king → queen)**
- **Plural (e.g., king → kings)**

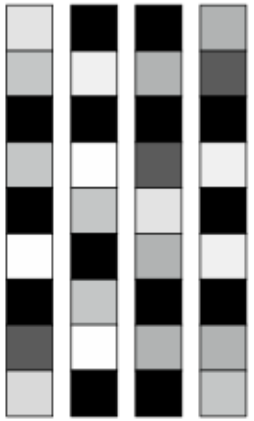
Word embeddings can have hundreds or thousands of such useful vectors

Understanding word embeddings

One-hot encoding vectors

- **Binary**
- **Sparse**
- **High-dimensional**
- **Hardcoded**





Word-embedding vectors

- **Floating point**
- **Dense**
- **Low-dimensional**
- **Learnt from data**

Sequence modelling approach

```
>>> model.summary()
Model: "lstm_with_embedding"
```

Layer (type)	Output Shape	Param #	Connected to
input_layer_3 (InputLayer)	(None, 600)	0	-
embedding (Embedding)	(None, 600, 64)	1,920,000	input_layer_6[0][…
not_equal (NotEqual)	(None, 600)	0	input_layer_6[0][…
bidirectional_1 (Bidirectional)	(None, 128)	66,048	embedding[0][0], not_equal[0][0]
dropout_1 (Dropout)	(None, 128)	0	bidirectional_2[0…
dense_3 (Dense)	(None, 1)	129	dropout_2[0][0]

Bidirectional LSTM layer

With pretrained embeddings, test accuracy ~89%

When to use deep learning

When to use deep learning?

- Deep learning has led to remarkable achievements
  - Image classification, text and image generation, translation, etc.
- That is not to say, it is the best tool for most problems
- Most of the time,
  - linear models will achieve similar or better results in tabular data
  - loss of interpretability and computational cost will not be worth it
- Deep learning is attractive when:
  - Dataset is extremely large
  - Interpretability is not high priority
  - Unstructured data: images, audio, text, etc.